CareOrbit Phase 1 MVP — Complete Technical Implementation (Revised)

Security-First, Web + Email, No WhatsApp, No Azure ML

PART 1: FOUNDATION

---

1. REVISED MVP SCOPE

What Changed From Previous Version

Item	Previous	Revised	Reason
WhatsApp integration	P0 (must-have)	**REMOVED** — Phase 2	Meta verification takes weeks, massive surface area for 2-person team
Authentication	Basic JWT	**Healthcare-grade auth** with refresh tokens, RBAC, field encryption, audit middleware	Health data demands it; judges will ask
Reminders	WhatsApp messages	**Email via Azure Communication Services**	Simpler, no third-party approval needed
Azure ML models	In architecture docs	**REMOVED** — Phase 3	No training data yet; GPT-4o + deterministic rules handle everything at MVP scale
Voice input (Speech)	P0	**REMOVED** — Phase 2	Only needed for WhatsApp voice notes; web can use text
Primary channel	WhatsApp-first	**Web-first**	Full control, richer UI, easier to demo

What's IN (Build This)

P0 — Must Ship:

- Prescription photo upload + AI extraction
- Medicine strip photo recognition
- Lab report photo extraction
- PHIG construction with FHIR R4 resources
- Deterministic confidence scoring system
- Medication Agent (drug interaction detection with severity modifiers)
- Care Gap Agent (guideline-based screening gap detection via RAG)
- Orchestrator (multi-agent routing + response synthesis)
- Health Summary PDF generation (single-page, confidence-annotated)
- Healthcare-grade authentication (refresh tokens, RBAC, audit trail)
- Patient confirmation flow (verify extracted data via web UI)
- Hindi + English support (Azure Translator)
- Email medication reminders
- Web dashboard (Next.js 14)

P1 — Should Ship If Time Allows:

- History Agent (health timeline narrative)
- Family caregiver view with permission levels
- Lab value trend visualization
- Medication adherence tracking (basic)

What's OUT (Don't Build Yet)

- WhatsApp integration (Phase 2)
- SMS notifications (Phase 2)
- Voice input / Azure Speech (Phase 2)
- FHIR API integration (Phase 2-3)
- Doctor Portal with QR scan (Phase 2)
- Azure ML predictive models (Phase 3 — needs 5,000+ patients for training data)
- ABDM / ABHA integration (Phase 2)
- Appointment Coordination Agent (Phase 2)
- Custom Vision models for document classification (Phase 3)
- Offline mode (Phase 3)
- Payment system (Phase 3)

MVP Success Metrics

Metric	Target
Prescription photo extraction accuracy	>80%
Drug interaction detection precision	>90%
Care gaps detected per chronic patient	2-4

End-to-end processing time	<30 seconds
Patient confirmation rate	>70%
Demo patients with full PHIG	3+ synthetic
Health summaries generated for demo	3+

---

2. AZURE INFRASTRUCTURE

2.1 Services to Provision (8 services — down from 11)

REMOVED from previous version:

- Azure AI Speech (was for WhatsApp voice notes — Phase 2)
- WhatsApp Business API setup
- Azure ML workspace

```
# ── Resource Group ──
az group create --name careorbit-mvp --location centralindia

# ── 1. Azure OpenAI (GPT-4o) ──
# Must use East US or Sweden Central (GPT-4o availability)
az cognitiveservices account create \
  --name careorbit-openai \
  --resource-group careorbit-mvp \
  --kind OpenAI \
  --sku S0 \
  --location eastus

# Deploy GPT-4o model
az cognitiveservices account deployment create \
  --name careorbit-openai \
  --resource-group careorbit-mvp \
  --deployment-name gpt-4o \
  --model-name gpt-4o \
  --model-version "2024-08-06" \
  --model-format OpenAI \
  --sku-name Standard \
  --sku-capacity 30

# ── 2. Azure AI Search (RAG knowledge base) ──
az search service create \
  --name careorbit-search \
```

```
  --resource-group careorbit-mvp \
  --sku basic \
  --location centralindia

# ── 3. Azure Database for PostgreSQL ──
az postgres flexible-server create \
  --name careorbit-db \
  --resource-group careorbit-mvp \
  --location centralindia \
  --tier Burstable \
  --sku-name Standard_B1ms \
  --storage-size 32 \
  --version 16 \
  --admin-user careorbit_admin \
  --admin-password '<STRONG_PASSWORD>' \
  --yes

# Enable required extensions
az postgres flexible-server parameter set \
  --name azure.extensions \
  --resource-group careorbit-mvp \
  --server-name careorbit-db \
  --value "UUID-OSSP,PGCRYPTO"

# Create database
az postgres flexible-server db create \
  --resource-group careorbit-mvp \
  --server-name careorbit-db \
  --database-name careorbit

# ── 4. Azure Blob Storage ──
az storage account create \
  --name careorbitstorage \
  --resource-group careorbit-mvp \
  --location centralindia \
  --sku Standard_LRS \
  --kind StorageV2

# Create containers
for container in prescription-uploads lab-reports medicine-strips health-summaries; do
  az storage container create \
    --name $container \
    --account-name careorbitstorage \
    --auth-mode login
done

# ── 5. Azure AI Vision (OCR) ──
az cognitiveservices account create \
```

```
  --name careorbit-vision \
  --resource-group careorbit-mvp \
  --kind ComputerVision \
  --sku S1 \
  --location centralindia

# ── 6. Azure AI Language (NER) ──
az cognitiveservices account create \
  --name careorbit-language \
  --resource-group careorbit-mvp \
  --kind TextAnalytics \
  --sku S \
  --location centralindia

# ── 7. Azure AI Translator ──
az cognitiveservices account create \
  --name careorbit-translator \
  --resource-group careorbit-mvp \
  --kind TextTranslation \
  --sku S1 \
  --location global

# ── 8. Azure App Service (Backend hosting) ──
az appservice plan create \
  --name careorbit-plan \
  --resource-group careorbit-mvp \
  --sku B1 \
  --is-linux

az webapp create \
  --name careorbit-api \
  --resource-group careorbit-mvp \
  --plan careorbit-plan \
  --runtime "PYTHON:3.11"

# ── 9. Azure Communication Services (Email reminders) ──
az communication create \
  --name careorbit-comms \
  --resource-group careorbit-mvp \
  --location global \
  --data-location india
```

2.2 Monthly Cost Estimate (Revised)

Service	Tier	Est. Cost
Azure OpenAI (GPT-4o)	S0, ~5K requests	~$15

Azure AI Search	Basic (fixed)	~$25
PostgreSQL Flexible	Burstable B1ms	~$13
Blob Storage	~10GB	~$1
Azure AI Vision	S1, ~3K OCR calls	~$5
Azure AI Language	S, ~3K NER calls	~$3
Azure AI Translator	S1, ~500K chars	~$3
App Service	B1 Linux	~$13
Azure Communication Services	Email, ~500 emails	~$1
TOTAL		**~$79/month**

Down from ~$83. Removed Speech ($5) and WhatsApp infra costs. Added email ($1).

---

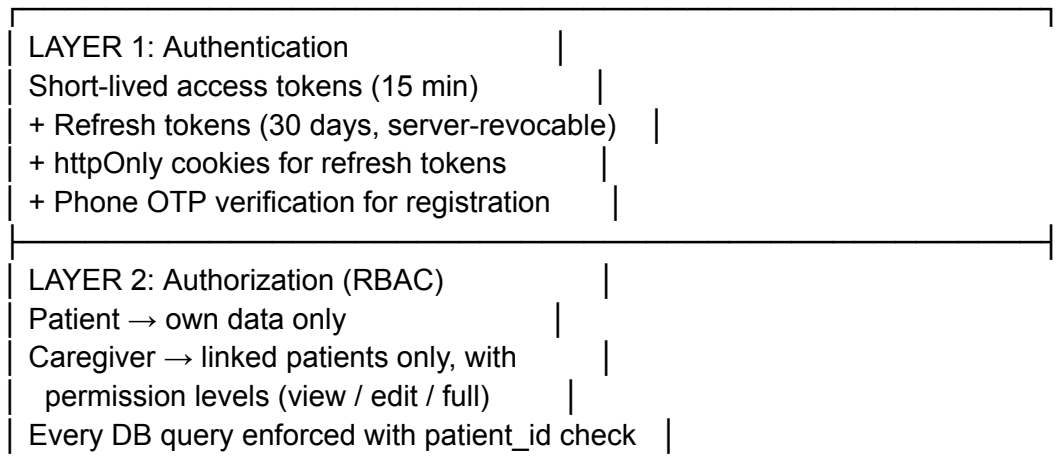
3. SECURITY ARCHITECTURE

3.1 Why Basic JWT Fails for Health Data

Basic JWT gives you ONE thing: proof someone logged in. It does not:

- Control which patient's data a user can see
- Expire fast enough to limit damage from token theft
- Track who accessed what (HIPAA audit requirement)
- Protect sensitive fields if the database is breached
- Differentiate patient vs caregiver access levels
- Revoke access when a caregiver relationship ends

3.2 Four-Layer Security Model



```
├──────────────────────────────────────────────────────────┤
│ LAYER 3: Audit Trail                                     │
│ Every data access logged: who, whose data,               │
│ what action, when, from where                            │
│ Tamper-resistant (append-only table)                     │
├──────────────────────────────────────────────────────────┤
│ LAYER 4: Encryption                                      │
│ At rest: Azure PostgreSQL AES-256 (automatic)            │
│ In transit: TLS 1.2+ (automatic via Azure)               │
│ Application-level: pgcrypto for PII fields               │
│ (phone, DOB, emergency contacts)                         │
└──────────────────────────────────────────────────────────┘
```

3.3 Complete Database Schema (Revised with Security)

```
-- ═══════════════════════════════════════════════════════
-- EXTENSIONS
-- ═══════════════════════════════════════════════════════
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ═══════════════════════════════════════════════════════
-- SECURITY: ENCRYPTION KEY MANAGEMENT
-- ═══════════════════════════════════════════════════════
-- Application-level encryption for PII fields.
-- The encryption key is stored in Azure Key Vault,
-- passed to the app via environment variable,
-- and NEVER stored in the database.
--
-- Usage pattern:
--   INSERT: pgp_sym_encrypt(phone, current_setting('app.encryption_key'))
--   SELECT: pgp_sym_decrypt(phone_encrypted, current_setting('app.encryption_key'))
--
-- For MVP, we encrypt: phone_number, date_of_birth, emergency_contact_phone
-- Everything else is protected by Azure's at-rest encryption + RBAC.

-- ═══════════════════════════════════════════════════════
-- TABLE: users
-- ═══════════════════════════════════════════════════════
CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email           VARCHAR(255) UNIQUE NOT NULL,
    phone_number    VARCHAR(20) UNIQUE,
    phone_encrypted BYTEA,                      -- pgcrypto encrypted phone
    password_hash   VARCHAR(255) NOT NULL,
    name            VARCHAR(255) NOT NULL,
    preferred_language VARCHAR(5) DEFAULT 'en',
```

```
    medical_literacy_level VARCHAR(20) DEFAULT 'standard',
    email_verified  BOOLEAN DEFAULT FALSE,
    is_active       BOOLEAN DEFAULT TRUE,
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_at      TIMESTAMPTZ DEFAULT NOW()
);

-- ═══════════════════════════════════════════════════════════
-- TABLE: refresh_tokens (Server-side token management)
-- ═══════════════════════════════════════════════════════════
-- Why this exists: Access tokens are short-lived (15 min).
-- When they expire, the client sends the refresh token
-- to get a new access token. If a refresh token is
-- compromised, we can revoke it from this table.
-- Basic JWT has NO revocation capability.
CREATE TABLE refresh_tokens (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_hash      VARCHAR(255) NOT NULL UNIQUE,   -- SHA-256 hash (never store raw
token)
    device_info     VARCHAR(500),                   -- "Chrome on Windows" for multi-device
    ip_address      INET,
    expires_at      TIMESTAMPTZ NOT NULL,
    revoked_at      TIMESTAMPTZ,                    -- NULL = active, set = revoked
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_refresh_tokens_user ON refresh_tokens(user_id) WHERE revoked_at
IS NULL;
CREATE INDEX idx_refresh_tokens_hash ON refresh_tokens(token_hash);

-- ═══════════════════════════════════════════════════════════
-- TABLE: patient_profiles
-- ═══════════════════════════════════════════════════════════
CREATE TABLE patient_profiles (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id             UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
    date_of_birth       DATE,
    dob_encrypted       BYTEA,              -- pgcrypto encrypted DOB
    gender              VARCHAR(20),
    blood_group         VARCHAR(10),
    height_cm           DECIMAL(5,1),
    weight_kg           DECIMAL(5,1),
    emergency_contact_name  VARCHAR(255),
    emergency_contact_phone_encrypted BYTEA,        -- pgcrypto encrypted
    city                VARCHAR(100),
    state               VARCHAR(100),
    country             VARCHAR(10) DEFAULT 'IN',
```

```
    created_at              TIMESTAMPTZ DEFAULT NOW(),
    updated_at              TIMESTAMPTZ DEFAULT NOW()
);

-- ═══════════════════════════════════════════════════════════
-- TABLE: caregiver_links (Authorization layer)
-- ═══════════════════════════════════════════════════════════
-- This table IS the authorization matrix.
-- When caregiver queries patient data, middleware checks:
--   1. Does a row exist linking this caregiver to this patient?
--   2. Is permission_level sufficient for the requested action?
--   3. Is the link still active?
CREATE TABLE caregiver_links (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    caregiver_user_id   UUID NOT NULL REFERENCES users(id),
    patient_user_id     UUID NOT NULL REFERENCES users(id),
    relationship        VARCHAR(50) NOT NULL,        -- son, daughter, spouse, parent, sibling
    permission_level    VARCHAR(20) NOT NULL DEFAULT 'view',
    -- view: see medications, conditions, labs, summary
    -- edit: upload documents, confirm data
    -- full: manage caregivers, generate summaries, manage reminders
    is_active           BOOLEAN DEFAULT TRUE,
    granted_at          TIMESTAMPTZ DEFAULT NOW(),
    revoked_at          TIMESTAMPTZ,
    UNIQUE(caregiver_user_id, patient_user_id)
);
CREATE INDEX idx_caregiver_links_caregiver ON caregiver_links(caregiver_user_id)
WHERE is_active = TRUE;
CREATE INDEX idx_caregiver_links_patient ON caregiver_links(patient_user_id) WHERE
is_active = TRUE;

-- ═══════════════════════════════════════════════════════════
-- TABLE: audit_log (Compliance layer — append-only)
-- ═══════════════════════════════════════════════════════════
-- EVERY data access gets logged here.
-- This table should NEVER have UPDATE or DELETE permissions
-- for application users — it's append-only for integrity.
CREATE TABLE audit_log (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id         UUID REFERENCES users(id),       -- Who performed the action
    patient_id      UUID REFERENCES users(id),       -- Whose data was accessed
    action          VARCHAR(50) NOT NULL,
    -- Actions: LOGIN, LOGOUT, VIEW_PROFILE, VIEW_MEDICATIONS,
    -- VIEW_LABS, VIEW_SUMMARY, UPLOAD_DOCUMENT, CONFIRM_DATA,
    -- GENERATE_SUMMARY, DOWNLOAD_PDF, CHAT_QUERY,
    -- ADD_CAREGIVER, REVOKE_CAREGIVER, UPDATE_PROFILE,
    -- EXPORT_DATA, DELETE_ACCOUNT
    resource_type   VARCHAR(50),                     -- phig_node, document, summary, profile
```

```
    resource_id     UUID,                       -- ID of the specific resource
    ip_address      INET,
    user_agent      VARCHAR(500),
    channel         VARCHAR(20) DEFAULT 'web',
    metadata        JSONB DEFAULT '{}',             -- Additional context
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_audit_patient ON audit_log(patient_id, created_at DESC);
CREATE INDEX idx_audit_user ON audit_log(user_id, created_at DESC);
CREATE INDEX idx_audit_action ON audit_log(action, created_at DESC);

-- ═══════════════════════════════════════════════════════════
-- PHIG TABLES (unchanged from previous version)
-- ═══════════════════════════════════════════════════════════

CREATE TABLE phig_nodes (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    patient_id          UUID NOT NULL REFERENCES users(id),
    node_type           VARCHAR(30) NOT NULL,
    display_name        VARCHAR(500) NOT NULL,
    display_name_hi     VARCHAR(500),
    clinical_code       VARCHAR(50),
    coding_system       VARCHAR(20),
    fhir_resource       JSONB NOT NULL DEFAULT '{}',
    confidence_score    DECIMAL(3,2) NOT NULL CHECK (confidence_score BETWEEN 0
AND 1),
    confidence_source   VARCHAR(50) NOT NULL,
    confidence_details  JSONB,
    source_document_id  UUID,
    is_active           BOOLEAN DEFAULT TRUE,
    effective_start     DATE,
    effective_end       DATE,
    created_at          TIMESTAMPTZ DEFAULT NOW(),
    updated_at          TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_phig_nodes_patient ON phig_nodes(patient_id, node_type) WHERE
is_active = TRUE;
CREATE INDEX idx_phig_nodes_clinical ON phig_nodes(patient_id, clinical_code) WHERE
is_active = TRUE;
CREATE INDEX idx_phig_nodes_confidence ON phig_nodes(patient_id, confidence_score
DESC);
CREATE INDEX idx_phig_nodes_fhir ON phig_nodes USING GIN (fhir_resource);

CREATE TABLE phig_edges (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    patient_id          UUID NOT NULL REFERENCES users(id),
    source_node_id      UUID NOT NULL REFERENCES phig_nodes(id),
    target_node_id      UUID NOT NULL REFERENCES phig_nodes(id),
```

```
    edge_type           VARCHAR(50) NOT NULL,
    properties          JSONB DEFAULT '{}',
    reasoning_chain     TEXT,
    confidence_score    DECIMAL(3,2),
    is_active           BOOLEAN DEFAULT TRUE,
    created_at          TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(source_node_id, target_node_id, edge_type)
);
CREATE INDEX idx_phig_edges_patient ON phig_edges(patient_id) WHERE is_active =
TRUE;
CREATE INDEX idx_phig_edges_source ON phig_edges(source_node_id) WHERE
is_active = TRUE;
CREATE INDEX idx_phig_edges_target ON phig_edges(target_node_id) WHERE is_active
= TRUE;
CREATE INDEX idx_phig_edges_props ON phig_edges USING GIN (properties);

-- ═══════════════════════════════════════════════════════════
-- DOCUMENT PROCESSING
-- ═══════════════════════════════════════════════════════════

CREATE TABLE uploaded_documents (
    id                      UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    patient_id              UUID NOT NULL REFERENCES users(id),
    uploaded_by             UUID NOT NULL REFERENCES users(id),
    document_type           VARCHAR(30) NOT NULL,
    document_type_confidence DECIMAL(3,2),
    original_blob_url       TEXT NOT NULL,
    ocr_raw_text            TEXT,
    ocr_confidence_avg      DECIMAL(3,2),
    extracted_data          JSONB DEFAULT '{}',
    processing_status       VARCHAR(20) NOT NULL DEFAULT 'pending',
    items_needing_confirmation JSONB DEFAULT '[]',
    patient_confirmed_at    TIMESTAMPTZ,
    upload_channel          VARCHAR(20) DEFAULT 'web',
    created_at              TIMESTAMPTZ DEFAULT NOW(),
    updated_at              TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_documents_patient ON uploaded_documents(patient_id, created_at
DESC);
CREATE INDEX idx_documents_status ON uploaded_documents(processing_status);

-- ═══════════════════════════════════════════════════════════
-- CONVERSATIONS & MESSAGES
-- ═══════════════════════════════════════════════════════════

CREATE TABLE conversations (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    patient_id      UUID NOT NULL REFERENCES users(id),
```

```
    channel         VARCHAR(20) DEFAULT 'web',
    started_at      TIMESTAMPTZ DEFAULT NOW(),
    last_message_at TIMESTAMPTZ DEFAULT NOW(),
    is_active       BOOLEAN DEFAULT TRUE
);
CREATE INDEX idx_conversations_patient ON conversations(patient_id, last_message_at
DESC);

CREATE TABLE messages (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    conversation_id     UUID NOT NULL REFERENCES conversations(id),
    patient_id          UUID NOT NULL REFERENCES users(id),
    role                VARCHAR(20) NOT NULL,          -- user, assistant, system
    content             TEXT NOT NULL,
    content_language    VARCHAR(5) DEFAULT 'en',
    agents_invoked      JSONB DEFAULT '[]',
    agent_reasoning     JSONB DEFAULT '{}',
    attachment_document_id UUID REFERENCES uploaded_documents(id),
    created_at          TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_messages_conversation ON messages(conversation_id, created_at);

-- ═══════════════════════════════════════════════════════════
-- EMAIL REMINDERS (Replaces WhatsApp reminders)
-- ═══════════════════════════════════════════════════════════

CREATE TABLE medication_reminders (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    patient_id          UUID NOT NULL REFERENCES users(id),
    medication_node_id  UUID NOT NULL REFERENCES phig_nodes(id),
    reminder_time       TIME NOT NULL,
    days_of_week        INTEGER[] DEFAULT '{1,2,3,4,5,6,7}',
    delivery_method     VARCHAR(20) DEFAULT 'email',   -- email only for MVP
    is_active           BOOLEAN DEFAULT TRUE,
    last_sent_at        TIMESTAMPTZ,
    last_confirmed_at   TIMESTAMPTZ,
    created_at          TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_reminders_patient ON medication_reminders(patient_id) WHERE
is_active = TRUE;
CREATE INDEX idx_reminders_schedule ON medication_reminders(reminder_time,
is_active) WHERE is_active = TRUE;

CREATE TABLE adherence_log (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    reminder_id         UUID NOT NULL REFERENCES medication_reminders(id),
    patient_id          UUID NOT NULL REFERENCES users(id),
    medication_node_id  UUID NOT NULL REFERENCES phig_nodes(id),
```

```
    scheduled_time      TIMESTAMPTZ NOT NULL,
    response            VARCHAR(20),                -- taken, skipped, snoozed, no_response
    responded_at        TIMESTAMPTZ,
    response_channel    VARCHAR(20) DEFAULT 'email',
    created_at          TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_adherence_patient ON adherence_log(patient_id, scheduled_time
DESC);

-- ═══════════════════════════════════════════════════════════
-- HEALTH SUMMARIES
-- ═══════════════════════════════════════════════════════════

CREATE TABLE health_summaries (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    patient_id          UUID NOT NULL REFERENCES users(id),
    generated_by        UUID NOT NULL REFERENCES users(id),
    summary_data        JSONB NOT NULL,
    pdf_blob_url        TEXT,
    verification_code   VARCHAR(8) UNIQUE DEFAULT
UPPER(LEFT(MD5(RANDOM()::TEXT), 8)),
    summary_type        VARCHAR(20) DEFAULT 'comprehensive',
    language            VARCHAR(5) DEFAULT 'en',
    generated_at        TIMESTAMPTZ DEFAULT NOW(),
    expires_at          TIMESTAMPTZ DEFAULT (NOW() + INTERVAL '90 days')
);
CREATE INDEX idx_summaries_patient ON health_summaries(patient_id, generated_at
DESC);
CREATE INDEX idx_summaries_verification ON health_summaries(verification_code);

-- ═══════════════════════════════════════════════════════════
-- USEFUL VIEWS
-- ═══════════════════════════════════════════════════════════

CREATE OR REPLACE VIEW v_active_medications AS
SELECT
    n.patient_id,
    n.id AS node_id,
    n.display_name,
    n.clinical_code AS rxnorm_code,
    n.confidence_score,
    n.confidence_source,
    n.effective_start,
    n.fhir_resource->'dosage'->0->>'text' AS dosage_text,
    n.fhir_resource->'informationSource'->>'display' AS prescriber,
    (SELECT COUNT(*) FROM phig_edges e
     WHERE (e.source_node_id = n.id OR e.target_node_id = n.id)
       AND e.edge_type = 'interacts_with' AND e.is_active = TRUE
```

```
    ) AS interaction_count
FROM phig_nodes n
WHERE n.node_type = 'medication' AND n.is_active = TRUE
ORDER BY n.confidence_score DESC;

CREATE OR REPLACE VIEW v_patient_health_overview AS
SELECT
    u.id AS patient_id,
    u.name,
    COUNT(*) FILTER (WHERE n.node_type = 'condition') AS conditions_count,
    COUNT(*) FILTER (WHERE n.node_type = 'medication') AS medications_count,
    COUNT(*) FILTER (WHERE n.node_type = 'lab_value') AS lab_values_count,
    COUNT(*) FILTER (WHERE n.node_type = 'care_gap') AS care_gaps_count,
    COUNT(*) FILTER (WHERE n.node_type = 'provider') AS providers_count
FROM users u
LEFT JOIN phig_nodes n ON n.patient_id = u.id AND n.is_active = TRUE
GROUP BY u.id, u.name;

-- ═══════════════════════════════════════════════════════════════
-- ROW LEVEL SECURITY (Patient data isolation)
-- ═══════════════════════════════════════════════════════════════
-- Note: RLS is enabled here but policies are enforced
-- via application middleware for MVP. Full RLS policies
-- would be added before production deployment.

ALTER TABLE phig_nodes ENABLE ROW LEVEL SECURITY;
ALTER TABLE phig_edges ENABLE ROW LEVEL SECURITY;
ALTER TABLE uploaded_documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE medication_reminders ENABLE ROW LEVEL SECURITY;
ALTER TABLE adherence_log ENABLE ROW LEVEL SECURITY;
ALTER TABLE health_summaries ENABLE ROW LEVEL SECURITY;
```

---

4. BACKEND PROJECT STRUCTURE

```
careorbit-backend/
├── main.py                    # FastAPI entry point
├── config.py                  # Pydantic settings
├── requirements.txt           # Dependencies
├── Dockerfile
├── .env.example
│
├── database/
│   ├── connection.py          # Async PostgreSQL pool
│   ├── schema.sql             # Complete schema (above)
│   ├── seed_guidelines.json   # Clinical guidelines for RAG
```

```
│   ├── seed_drug_interactions.json       # Drug interactions for RAG
│   └── seed_demo.py                      # Synthetic demo patients
│
├── api/
│   ├── routes/
│   │   ├── auth.py                  # Registration, login, token refresh
│   │   ├── patients.py              # Profile, overview, alerts
│   │   ├── documents.py             # Document upload
│   │   ├── chat.py                  # AI chat queries
│   │   ├── summary.py               # Health summary PDF
│   │   ├── caregivers.py            # Caregiver management
│   │   └── reminders.py             # Email reminder management
│   └── middleware/
│       ├── auth.py                  # JWT + refresh token verification
│       ├── rbac.py                  # Role-based access control
│       └── audit.py                 # Audit trail middleware
│
├── pipeline/
│   ├── document_classifier.py
│   ├── prescription_extractor.py
│   ├── lab_report_extractor.py
│   ├── medicine_strip_reader.py
│   ├── fhir_converter.py
│   └── document_pipeline.py         # Master orchestrator
│
├── graph/
│   ├── confidence.py                # Deterministic scoring
│   └── phig_builder.py              # Graph CRUD + analysis
│
├── agents/
│   ├── base_agent.py
│   ├── medication_agent.py
│   ├── care_gap_agent.py
│   ├── history_agent.py
│   └── orchestrator.py
│
├── services/
│   ├── azure_openai.py
│   ├── azure_vision.py
│   ├── azure_blob.py
│   ├── azure_search.py
│   ├── azure_translator.py
│   └── email_service.py             # Azure Communication Services
│
├── utils/
│   ├── drug_database.py
│   ├── summary_generator.py
│   └── encryption.py                # PII field encryption helpers
```

```
│
└── tests/
    ├── conftest.py
    ├── test_confidence.py
    ├── test_drug_database.py
    ├── test_auth.py                # Auth + RBAC tests
    ├── test_pipeline_e2e.py
    └── test_agents.py
```

4.1 requirements.txt

```
# Core Framework
fastapi==0.109.2
uvicorn[standard]==0.27.1
python-multipart==0.0.9
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
pydantic==2.6.1
pydantic-settings==2.1.0

# Database
asyncpg==0.29.0
sqlalchemy[asyncio]==2.0.27
alembic==1.13.1

# Azure AI Services
openai==1.12.0
azure-ai-textanalytics==5.3.0
azure-ai-vision-imageanalysis==1.0.0b3
azure-ai-translation-text==1.0.0b1
azure-search-documents==11.4.0
azure-storage-blob==12.19.0

# Azure Communication Services (Email)
azure-communication-email==1.0.0

# HTTP
httpx==0.27.0

# PDF Generation
reportlab==4.1.0

# Utilities
python-dotenv==1.0.1
Pillow==10.2.0
python-dateutil==2.8.2
```

```
# Development
pytest==8.0.1
pytest-asyncio==0.23.5
```

4.2 Configuration

```
# config.py

from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    ENVIRONMENT: str = "development"

    # ── Security ──
    JWT_SECRET: str
    JWT_ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRY_MINUTES: int = 15        # Short-lived
    REFRESH_TOKEN_EXPIRY_DAYS: int = 30          # Long-lived, revocable
    ENCRYPTION_KEY: str                          # For PII field encryption
    CORS_ORIGINS: str = "http://localhost:3000"

    # ── Database ──
    DATABASE_URL: str

    # ── Azure OpenAI ──
    AZURE_OPENAI_ENDPOINT: str
    AZURE_OPENAI_API_KEY: str
    AZURE_OPENAI_DEPLOYMENT: str = "gpt-4o"
    AZURE_OPENAI_API_VERSION: str = "2024-08-06"

    # ── Azure AI Search ──
    AZURE_SEARCH_ENDPOINT: str
    AZURE_SEARCH_API_KEY: str
    AZURE_SEARCH_INDEX_GUIDELINES: str = "clinical-guidelines"
    AZURE_SEARCH_INDEX_DRUGS: str = "drug-interactions"

    # ── Azure Blob Storage ──
    AZURE_STORAGE_CONNECTION_STRING: str
    AZURE_STORAGE_ACCOUNT_NAME: str

    # ── Azure AI Vision ──
    AZURE_VISION_ENDPOINT: str
    AZURE_VISION_API_KEY: str

    # ── Azure AI Language ──
    AZURE_LANGUAGE_ENDPOINT: str
```

```
    AZURE_LANGUAGE_API_KEY: str

    # ── Azure AI Translator ──
    AZURE_TRANSLATOR_API_KEY: str
    AZURE_TRANSLATOR_REGION: str = "global"
    AZURE_TRANSLATOR_ENDPOINT: str = "https://api.cognitive.microsofttranslator.com/"

    # ── Azure Communication Services (Email) ──
    AZURE_COMM_CONNECTION_STRING: str
    AZURE_COMM_SENDER_EMAIL: str = "DoNotReply@careorbit.com"

    class Config:
        env_file = ".env"
        case_sensitive = True

@lru_cache()
def get_settings() -> Settings:
    return Settings()
```

4.3 FastAPI Entry Point

```
# main.py

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
from config import get_settings
from database.connection import init_db, close_db
from api.routes import auth, patients, documents, chat, summary, caregivers, reminders

@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    yield
    await close_db()

settings = get_settings()
app = FastAPI(title="CareOrbit API", version="0.2.0", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS.split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
app.include_router(auth.router, prefix="/api/auth", tags=["Authentication"])
app.include_router(patients.router, prefix="/api/patients", tags=["Patients"])
app.include_router(documents.router, prefix="/api/documents", tags=["Documents"])
app.include_router(chat.router, prefix="/api/chat", tags=["AI Chat"])
app.include_router(summary.router, prefix="/api/summary", tags=["Health Summary"])
app.include_router(caregivers.router, prefix="/api/caregivers", tags=["Caregivers"])
app.include_router(reminders.router, prefix="/api/reminders", tags=["Reminders"])

@app.get("/health")
async def health_check():
    return {"status": "healthy", "service": "careorbit-api", "version": "0.2.0"}
```

4.4 Database Connection

```
# database/connection.py

from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession,
async_sessionmaker
from config import get_settings

settings = get_settings()
async_database_url = settings.DATABASE_URL.replace("postgresql://",
"postgresql+asyncpg://")

engine = create_async_engine(
    async_database_url,
    pool_size=10, max_overflow=20,
    pool_timeout=30, pool_recycle=1800,
    echo=settings.ENVIRONMENT == "development"
)

async_session = async_sessionmaker(engine, class_=AsyncSession,
expire_on_commit=False)

async def init_db():
    async with engine.begin() as conn:
        # Set encryption key for this connection pool
        await conn.execute(
            f"SET app.encryption_key = '{settings.ENCRYPTION_KEY}'"
        )

async def close_db():
    await engine.dispose()

async def get_db() -> AsyncSession:
    async with async_session() as session:
        try:
```

```
        # Set encryption key for each session
        await session.execute(
            f"SET LOCAL app.encryption_key = '{settings.ENCRYPTION_KEY}'"
        )
        yield session
    finally:
        await session.close()
```

PART 2: SECURITY, AZURE CLIENTS & PIPELINE

---

5. HEALTHCARE-GRADE AUTHENTICATION

5.1 Auth Service (Token Management)

```
# api/middleware/auth.py

from fastapi import Depends, HTTPException, status, Request
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from jose import jwt, JWTError
from passlib.context import CryptContext
from config import get_settings
from database.connection import async_session
from sqlalchemy import text
from datetime import datetime, timedelta
from hashlib import sha256
import secrets

settings = get_settings()
security = HTTPBearer()
pwd_context = CryptContext(schemes=["bcrypt"])


def hash_password(password: str) -> str:
    return pwd_context.hash(password)


def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)


def create_access_token(user_id: str) -> str:
```

```
    """
    Short-lived access token (15 minutes).
    If stolen, damage window is 15 minutes max.
    """
    expire = datetime.utcnow() +
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRY_MINUTES)
    payload = {
        "sub": user_id,
        "type": "access",
        "exp": expire,
        "iat": datetime.utcnow()
    }
    return jwt.encode(payload, settings.JWT_SECRET,
algorithm=settings.JWT_ALGORITHM)


async def create_refresh_token(user_id: str, ip_address: str = None,
                               device_info: str = None) -> str:
    """
    Long-lived refresh token (30 days).
    Stored server-side as SHA-256 hash — can be revoked at any time.
    """
    raw_token = secrets.token_urlsafe(64)
    token_hash = sha256(raw_token.encode()).hexdigest()
    expires_at = datetime.utcnow() +
timedelta(days=settings.REFRESH_TOKEN_EXPIRY_DAYS)

    async with async_session() as session:
        await session.execute(text("""
            INSERT INTO refresh_tokens (user_id, token_hash, device_info, ip_address,
expires_at)
            VALUES (:uid, :hash, :device, :ip, :expires)
        """), {
            "uid": user_id, "hash": token_hash,
            "device": device_info, "ip": ip_address,
            "expires": expires_at
        })
        await session.commit()

    return raw_token


async def verify_refresh_token(raw_token: str) -> dict:
    """Verify a refresh token and return user info."""
    token_hash = sha256(raw_token.encode()).hexdigest()

    async with async_session() as session:
        result = await session.execute(text("""
```

```
            SELECT rt.user_id, rt.expires_at, u.name, u.email
            FROM refresh_tokens rt
            JOIN users u ON u.id = rt.user_id
            WHERE rt.token_hash = :hash
              AND rt.revoked_at IS NULL
              AND rt.expires_at > NOW()
        """), {"hash": token_hash})
        row = result.mappings().first()

    if not row:
        raise HTTPException(status_code=401, detail="Invalid or expired refresh token")

    return dict(row)


async def revoke_refresh_token(raw_token: str):
    """Revoke a refresh token (logout, compromise response)."""
    token_hash = sha256(raw_token.encode()).hexdigest()
    async with async_session() as session:
        await session.execute(text("""
            UPDATE refresh_tokens SET revoked_at = NOW()
            WHERE token_hash = :hash
        """), {"hash": token_hash})
        await session.commit()


async def revoke_all_user_tokens(user_id: str):
    """Revoke ALL refresh tokens for a user (password change, security event)."""
    async with async_session() as session:
        await session.execute(text("""
            UPDATE refresh_tokens SET revoked_at = NOW()
            WHERE user_id = :uid AND revoked_at IS NULL
        """), {"uid": user_id})
        await session.commit()


async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security)
) -> dict:
    """
    Verify access token and return current user.
    Used as FastAPI dependency on protected routes.
    """
    try:
        payload = jwt.decode(
            credentials.credentials,
            settings.JWT_SECRET,
            algorithms=[settings.JWT_ALGORITHM]
```

```
        )
        if payload.get("type") != "access":
            raise HTTPException(status_code=401, detail="Invalid token type")

        user_id = payload.get("sub")
        if not user_id:
            raise HTTPException(status_code=401, detail="Invalid token")

        return {"id": user_id}

    except JWTError:
        raise HTTPException(status_code=401, detail="Token expired or invalid")
```

5.2 RBAC Middleware (Who Can See Whose Data)

```
# api/middleware/rbac.py

from fastapi import HTTPException
from database.connection import async_session
from sqlalchemy import text


async def verify_patient_access(requesting_user_id: str, target_patient_id: str,
                                required_permission: str = "view") -> dict:
    """
    Core authorization check. Called before ANY patient data access.

    Returns access context:
    {
        "access_type": "self" | "caregiver",
        "permission_level": "view" | "edit" | "full",
        "relationship": "self" | "son" | "daughter" | ...
    }

    Raises 403 if access denied.
    """
    # Case 1: Patient accessing their own data — always allowed
    if requesting_user_id == target_patient_id:
        return {
            "access_type": "self",
            "permission_level": "full",
            "relationship": "self"
        }

    # Case 2: Caregiver — check caregiver_links table
    async with async_session() as session:
        result = await session.execute(text("""
```

```
            SELECT permission_level, relationship
            FROM caregiver_links
            WHERE caregiver_user_id = :caregiver
              AND patient_user_id = :patient
              AND is_active = TRUE
              AND revoked_at IS NULL
        """), {"caregiver": requesting_user_id, "patient": target_patient_id})
        link = result.mappings().first()

    if not link:
        raise HTTPException(
            status_code=403,
            detail="You do not have permission to access this patient's data."
        )

    # Check permission level
    permission_hierarchy = {"view": 0, "edit": 1, "full": 2}
    user_level = permission_hierarchy.get(link["permission_level"], 0)
    required_level = permission_hierarchy.get(required_permission, 0)

    if user_level < required_level:
        raise HTTPException(
            status_code=403,
            detail=f"Your permission level ({link['permission_level']}) "
                   f"is insufficient. This action requires '{required_permission}'."
        )

    return {
        "access_type": "caregiver",
        "permission_level": link["permission_level"],
        "relationship": link["relationship"]
    }
```

5.3 Audit Middleware

```
# api/middleware/audit.py

from database.connection import async_session
from sqlalchemy import text
from fastapi import Request
import json


async def log_audit(
    user_id: str,
    patient_id: str,
    action: str,
```

```
    resource_type: str = None,
    resource_id: str = None,
    request: Request = None,
    metadata: dict = None
):
    """
    Log every data access to the audit trail.
    This is append-only — no updates, no deletes.
    """
    ip_address = None
    user_agent = None

    if request:
        ip_address = request.client.host if request.client else None
        user_agent = request.headers.get("User-Agent", "")[:500]

    async with async_session() as session:
        await session.execute(text("""
            INSERT INTO audit_log (
                user_id, patient_id, action, resource_type,
                resource_id, ip_address, user_agent, channel, metadata
            ) VALUES (
                :uid, :pid, :action, :rtype,
                :rid, :ip, :ua, 'web', :meta
            )
        """), {
            "uid": user_id,
            "pid": patient_id,
            "action": action,
            "rtype": resource_type,
            "rid": resource_id,
            "ip": ip_address,
            "ua": user_agent,
            "meta": json.dumps(metadata or {})
        })
        await session.commit()
```

5.4 PII Encryption Helpers

```
# utils/encryption.py

from config import get_settings

settings = get_settings()


def encrypt_sql(field_value: str) -> str:
```

```
    """
    Returns SQL expression for encrypting a value.
    Usage in raw SQL: pgp_sym_encrypt(:phone, :key)
    """
    return field_value  # In execute(), pass encryption_key as param


def get_encryption_params(values: dict) -> dict:
    """Add encryption key to SQL params."""
    values["encryption_key"] = settings.ENCRYPTION_KEY
    return values


# Example usage in a route:
#
# await session.execute(text("""
#     INSERT INTO users (phone_encrypted, ...)
#     VALUES (pgp_sym_encrypt(:phone, :encryption_key), ...)
# """), get_encryption_params({"phone": "+919876543210"}))
#
# await session.execute(text("""
#     SELECT pgp_sym_decrypt(phone_encrypted, :encryption_key) as phone
#     FROM users WHERE id = :id
# """), get_encryption_params({"id": user_id}))
```

5.5 Auth Routes

```
# api/routes/auth.py

from fastapi import APIRouter, HTTPException, Request, Response
from pydantic import BaseModel, EmailStr
from database.connection import async_session
from api.middleware.auth import (
    hash_password, verify_password, create_access_token,
    create_refresh_token, verify_refresh_token,
    revoke_refresh_token, revoke_all_user_tokens
)
from api.middleware.audit import log_audit
from utils.encryption import get_encryption_params
from sqlalchemy import text

router = APIRouter()


class RegisterRequest(BaseModel):
    email: EmailStr
    password: str        # Min 8 chars, enforced client-side + here
```

```
    name: str
    phone_number: str = None
    preferred_language: str = "en"


class LoginRequest(BaseModel):
    email: EmailStr
    password: str


class RefreshRequest(BaseModel):
    refresh_token: str


@router.post("/register")
async def register(req: RegisterRequest, request: Request):
    if len(req.password) < 8:
        raise HTTPException(400, "Password must be at least 8 characters")

    async with async_session() as session:
        # Check email uniqueness
        existing = await session.execute(
            text("SELECT id FROM users WHERE email = :email"),
            {"email": req.email}
        )
        if existing.first():
            raise HTTPException(400, "Email already registered")

        # Create user with encrypted PII
        from uuid import uuid4
        user_id = str(uuid4())
        params = get_encryption_params({
            "id": user_id,
            "email": req.email,
            "password_hash": hash_password(req.password),
            "name": req.name,
            "phone": req.phone_number,
            "lang": req.preferred_language
        })

        await session.execute(text("""
            INSERT INTO users (id, email, password_hash, name, phone_number,
                               phone_encrypted, preferred_language)
            VALUES (:id, :email, :password_hash, :name, :phone,
                    CASE WHEN :phone IS NOT NULL
                        THEN pgp_sym_encrypt(:phone, :encryption_key)
                        ELSE NULL END,
                    :lang)
```

```
        """), params)

        # Create empty patient profile
        await session.execute(text("""
            INSERT INTO patient_profiles (user_id) VALUES (:uid)
        """), {"uid": user_id})

        await session.commit()

    # Generate tokens
    access_token = create_access_token(user_id)
    refresh_token = await create_refresh_token(
        user_id,
        ip_address=request.client.host if request.client else None,
        device_info=request.headers.get("User-Agent", "")[:200]
    )

    await log_audit(user_id, user_id, "REGISTER", request=request)

    return {
        "access_token": access_token,
        "refresh_token": refresh_token,
        "token_type": "bearer",
        "user": {"id": user_id, "name": req.name, "email": req.email}
    }


@router.post("/login")
async def login(req: LoginRequest, request: Request):
    async with async_session() as session:
        result = await session.execute(
            text("SELECT id, password_hash, name, email FROM users WHERE email = :email
AND is_active = TRUE"),
            {"email": req.email}
        )
        user = result.mappings().first()

    if not user or not verify_password(req.password, user["password_hash"]):
        raise HTTPException(401, "Invalid email or password")

    user_id = str(user["id"])
    access_token = create_access_token(user_id)
    refresh_token = await create_refresh_token(
        user_id,
        ip_address=request.client.host if request.client else None,
        device_info=request.headers.get("User-Agent", "")[:200]
    )
```

```
    await log_audit(user_id, user_id, "LOGIN", request=request)

    return {
        "access_token": access_token,
        "refresh_token": refresh_token,
        "token_type": "bearer",
        "user": {"id": user_id, "name": user["name"], "email": user["email"]}
    }


@router.post("/refresh")
async def refresh_tokens(req: RefreshRequest, request: Request):
    """Exchange refresh token for new access token."""
    user_info = await verify_refresh_token(req.refresh_token)
    user_id = str(user_info["user_id"])

    # Rotate: revoke old refresh token, issue new one
    await revoke_refresh_token(req.refresh_token)

    new_access = create_access_token(user_id)
    new_refresh = await create_refresh_token(
        user_id,
        ip_address=request.client.host if request.client else None,
        device_info=request.headers.get("User-Agent", "")[:200]
    )

    return {
        "access_token": new_access,
        "refresh_token": new_refresh,
        "token_type": "bearer"
    }


@router.post("/logout")
async def logout(req: RefreshRequest, request: Request):
    """Revoke refresh token on logout."""
    try:
        user_info = await verify_refresh_token(req.refresh_token)
        await revoke_refresh_token(req.refresh_token)
        await log_audit(str(user_info["user_id"]), str(user_info["user_id"]),
                        "LOGOUT", request=request)
    except Exception:
        pass  # Already expired/revoked — that's fine
    return {"status": "logged out"}
```

---

6. AZURE SERVICE CLIENTS

Azure OpenAI, Vision, Blob, Search, Translator are identical to previous version. Only change: WhatsApp service REMOVED, Email service ADDED.

6.1 Azure OpenAI Client

```
# services/azure_openai.py — (same as previous version)

from openai import AsyncAzureOpenAI
from config import get_settings
import json

settings = get_settings()

class AzureOpenAIService:
    def __init__(self):
        self.client = AsyncAzureOpenAI(
            azure_endpoint=settings.AZURE_OPENAI_ENDPOINT,
            api_key=settings.AZURE_OPENAI_API_KEY,
            api_version=settings.AZURE_OPENAI_API_VERSION
        )
        self.deployment = settings.AZURE_OPENAI_DEPLOYMENT

    async def chat(self, system_prompt: str, user_message: str,
                   temperature: float = 0.3, max_tokens: int = 2000,
                   response_format: str = None) -> str:
        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message}
        ]
        kwargs = {
            "model": self.deployment, "messages": messages,
            "temperature": temperature, "max_tokens": max_tokens,
        }
        if response_format == "json_object":
            kwargs["response_format"] = {"type": "json_object"}
        response = await self.client.chat.completions.create(**kwargs)
        return response.choices[0].message.content

    async def chat_with_history(self, system_prompt: str, messages: list,
                                temperature: float = 0.4, max_tokens: int = 2000) -> str:
        full_messages = [{"role": "system", "content": system_prompt}]
        full_messages.extend(messages)
        response = await self.client.chat.completions.create(
            model=self.deployment, messages=full_messages,
            temperature=temperature, max_tokens=max_tokens
        )
```

```
        return response.choices[0].message.content

    async def extract_structured_data(self, system_prompt: str,
                                      user_message: str, temperature: float = 0.1) -> dict:
        response = await self.chat(
            system_prompt=system_prompt, user_message=user_message,
            temperature=temperature, response_format="json_object"
        )
        try:
            return json.loads(response)
        except json.JSONDecodeError:
            cleaned = response.strip()
            if cleaned.startswith("```json"): cleaned = cleaned[7:]
            if cleaned.endswith("```"): cleaned = cleaned[:-3]
            return json.loads(cleaned.strip())

openai_service = AzureOpenAIService()
```

6.2 Azure Vision, Blob, Search, Translator

(These are identical to previous Part 2 — azure_vision.py, azure_blob.py, azure_search.py, azure_translator.py. Refer to previous document for full code. No changes.)

6.3 Email Service (NEW — Replaces WhatsApp)

```
# services/email_service.py — Azure Communication Services Email

from azure.communication.email import EmailClient
from config import get_settings
from typing import Optional

settings = get_settings()


class EmailService:
    """
    Email notifications via Azure Communication Services.
    Replaces WhatsApp for MVP. Used for:
    - Medication reminders
    - Upload confirmation results
    - Drug interaction alerts
    - Care gap notifications
    """

    def __init__(self):
        self.client = EmailClient.from_connection_string(
            settings.AZURE_COMM_CONNECTION_STRING
```

```
        )
        self.sender = settings.AZURE_COMM_SENDER_EMAIL

    async def send_medication_reminder(self, to_email: str, patient_name: str,
                                       medication_name: str, dosage: str,
                                       time_label: str):
        """Send a medication reminder email."""
        subject = f"CareOrbit Reminder: Time for {medication_name}"
        body_html = f"""
        <div style="font-family: Arial, sans-serif; max-width: 500px; margin: 0 auto;">
          <h2 style="color: #1a5276;">💊 Medication Reminder</h2>
          <p>Hi {patient_name},</p>
          <p>It's time for your <strong>{time_label}</strong> medication:</p>
          <div style="background: #eaf2f8; padding: 15px; border-radius: 8px; margin: 15px
0;">
            <p style="font-size: 18px; margin: 0;"><strong>{medication_name}</strong></p>
            <p style="color: #666; margin: 5px 0 0 0;">{dosage}</p>
          </div>
          <p>
            <a href="{{CONFIRM_URL}}" style="background: #27ae60; color: white;
              padding: 10px 25px; border-radius: 5px; text-decoration: none;">
              ✅ Taken</a>
              
            <a href="{{SKIP_URL}}" style="background: #e74c3c; color: white;
              padding: 10px 25px; border-radius: 5px; text-decoration: none;">
              ⏭️ Skipped</a>
          </p>
          <p style="color: #999; font-size: 12px; margin-top: 30px;">
            CareOrbit — Your AI Health Coordinator
          </p>
        </div>
        """

        await self._send(to_email, subject, body_html)

    async def send_interaction_alert(self, to_email: str, patient_name: str,
                                     drug_a: str, drug_b: str,
                                     severity: str, description: str,
                                     clinical_action: str):
        """Send a drug interaction alert email."""
        severity_color = "#e74c3c" if severity in ("High", "CRITICAL", "ELEVATED") else
"#f39c12"
        subject = f"⚠️ CareOrbit Alert: Drug Interaction Detected"
        body_html = f"""
        <div style="font-family: Arial, sans-serif; max-width: 500px; margin: 0 auto;">
          <h2 style="color: {severity_color};">⚠️ Drug Interaction Alert</h2>
          <p>Hi {patient_name},</p>
          <p>CareOrbit detected a potential interaction between your medications:</p>
```

```
        <div style="background: #fdf2f2; padding: 15px; border-radius: 8px;
                    border-left: 4px solid {severity_color}; margin: 15px 0;">
          <p><strong>{drug_a}</strong> + <strong>{drug_b}</strong></p>
          <p>Severity: <strong style="color: {severity_color};">{severity}</strong></p>
          <p>{description}</p>
        </div>
        <p><strong>Recommended action:</strong> {clinical_action}</p>
        <p>⚕️ <em>Please discuss this with your doctor at your next visit.</em></p>
      </div>
      """

      await self._send(to_email, subject, body_html)

  async def send_upload_result(self, to_email: str, patient_name: str,
                               document_type: str, medications_found: int,
                               interactions_found: int, care_gaps_found: int):
      """Send document processing results email."""
      subject = f"CareOrbit: Your {document_type} has been processed"
      body_html = f"""
      <div style="font-family: Arial, sans-serif; max-width: 500px; margin: 0 auto;">
        <h2 style="color: #1a5276;">📋 Document Processed</h2>
        <p>Hi {patient_name},</p>
        <p>Your {document_type} has been analyzed. Here's what we found:</p>
        <ul>
          <li>💊 <strong>{medications_found}</strong> medications extracted</li>
          <li>⚠️ <strong>{interactions_found}</strong> interaction(s) detected</li>
          <li>📋 <strong>{care_gaps_found}</strong> care gap(s) identified</li>
        </ul>
        <p><a href="{{DASHBOARD_URL}}" style="background: #2980b9; color: white;
          padding: 10px 25px; border-radius: 5px; text-decoration: none;">
          View Details</a></p>
      </div>
      """

      await self._send(to_email, subject, body_html)

  async def _send(self, to_email: str, subject: str, body_html: str):
      """Send an email via Azure Communication Services."""
      message = {
          "senderAddress": self.sender,
          "recipients": {
              "to": [{"address": to_email}]
          },
          "content": {
              "subject": subject,
              "html": body_html
          }
      }
```

```
        # Azure Communication Services email send
        poller = self.client.begin_send(message)
        poller.result()  # Wait for completion


email_service = EmailService()
```

---

7. CONFIDENCE SCORING & PIPELINE

Confidence calculator, document classifier, prescription extractor, lab report extractor, medicine strip reader, drug database, FHIR converter are identical to previous Part 2.

Only change in the master pipeline: WhatsApp references removed, email notification added.

7.1 Master Document Pipeline (Revised)

```
# pipeline/document_pipeline.py — Master pipeline (WhatsApp removed, email added)

from pipeline.document_classifier import document_classifier
from pipeline.prescription_extractor import prescription_extractor
from pipeline.lab_report_extractor import lab_report_extractor
from pipeline.medicine_strip_reader import medicine_strip_reader
from pipeline.fhir_converter import FHIRConverter
from graph.confidence import ConfidenceCalculator
from graph.phig_builder import phig_builder
from services.azure_blob import blob_service
from services.azure_vision import vision_service
from services.email_service import email_service
from api.middleware.audit import log_audit
from dataclasses import dataclass, field
from typing import Optional, Union
from datetime import datetime
import uuid


@dataclass
class DocumentProcessingResult:
    document_id: str
    document_type: str
    processing_status: str
    extracted_data: Optional[object] = None
    nodes_created: list = field(default_factory=list)
    edges_created: list = field(default_factory=list)
    interaction_alerts: list = field(default_factory=list)
    care_gap_alerts: list = field(default_factory=list)
```

```
    confirmation_needed: list = field(default_factory=list)
    error_message: Optional[str] = None
    processing_time_ms: int = 0
    blob_url: Optional[str] = None


class DocumentPipeline:
    """
    Master pipeline: Photo → Classify → Extract → FHIR → PHIG → Analyze → Notify.
    Single entry point for all document uploads (web only for MVP).
    """

    async def process_document(
        self,
        image_bytes: bytes,
        patient_id: str,
        uploaded_by: str,
        patient_email: str = None,
        patient_name: str = None,
        file_extension: str = "jpg"
    ) -> DocumentProcessingResult:

        start_time = datetime.utcnow()
        doc_id = str(uuid.uuid4())

        try:
            # Stage 1: Upload to Blob Storage
            blob_url = await blob_service.upload_document(
                file_bytes=image_bytes,
                patient_id=patient_id,
                document_type="pending",
                file_extension=file_extension
            )

            # Stage 2: Classify Document Type
            classification = await document_classifier.classify(image_bytes)

            if classification.document_type in ("unreadable", "non_medical"):
                error_msg = (
                    classification.reupload_reason
                    if classification.document_type == "unreadable"
                    else "This doesn't appear to be a medical document."
                )
                return DocumentProcessingResult(
                    document_id=doc_id,
                    document_type=classification.document_type,
                    processing_status="failed",
                    error_message=error_msg,
```

```
                blob_url=blob_url,
                processing_time_ms=self._elapsed_ms(start_time)
            )

        # Stage 3: OCR
        ocr_result = await vision_service.extract_text(image_bytes)

        # Stage 4: Route to extractor
        result = DocumentProcessingResult(
            document_id=doc_id,
            document_type=classification.document_type,
            processing_status="processing",
            blob_url=blob_url
        )

        if classification.document_type == "prescription":
            await self._process_prescription(result, image_bytes, ocr_result, patient_id,
doc_id)
        elif classification.document_type == "lab_report":
            await self._process_lab_report(result, image_bytes, ocr_result, patient_id, doc_id)
        elif classification.document_type == "medicine_strip":
            await self._process_medicine_strip(result, image_bytes, ocr_result, patient_id,
doc_id)
        else:
            result.processing_status = "needs_confirmation"
            result.confirmation_needed = [{
                "type": "generic_document",
                "message": "We detected a medical document but can't extract structured "
                           "data automatically. Please tell us what's in this document."
            }]

        # Stage 5: Cascading analysis
        if result.nodes_created:
            medication_nodes = [n for n in result.nodes_created if n.get("node_type") ==
"medication"]
            for med_node in medication_nodes:
                interactions = await phig_builder.check_interactions_for_node(
                    patient_id=patient_id, medication_node_id=med_node["id"]
                )
                result.interaction_alerts.extend(interactions)

            care_gaps = await phig_builder.evaluate_care_gaps(patient_id)
            result.care_gap_alerts = care_gaps

        # Stage 6: Email notification (replaces WhatsApp)
        if patient_email and result.processing_status in ("success", "needs_confirmation"):
            try:
                await email_service.send_upload_result(
```

```
                    to_email=patient_email,
                    patient_name=patient_name or "Patient",
                    document_type=classification.document_type,
                    medications_found=len([n for n in result.nodes_created if n.get("node_type")
== "medication"]),
                    interactions_found=len(result.interaction_alerts),
                    care_gaps_found=len(result.care_gap_alerts)
                )

                # Send individual interaction alerts via email
                for alert in result.interaction_alerts:
                    if alert.get("alert_level") in ("URGENT_ALERT", "WARNING"):
                        await email_service.send_interaction_alert(
                            to_email=patient_email,
                            patient_name=patient_name or "Patient",
                            drug_a=alert["drug_a"],
                            drug_b=alert["drug_b"],
                            severity=alert["severity"],
                            description=alert["description"],
                            clinical_action=alert["clinical_action"]
                        )
            except Exception:
                pass  # Email failure shouldn't block processing

        # Audit log
        await log_audit(
            user_id=uploaded_by, patient_id=patient_id,
            action="UPLOAD_DOCUMENT", resource_type="document",
            resource_id=doc_id,
            metadata={
                "document_type": classification.document_type,
                "nodes_created": len(result.nodes_created),
                "interactions_found": len(result.interaction_alerts)
            }
        )

        # Set final status
        if result.confirmation_needed:
            result.processing_status = "needs_confirmation"
        elif not result.error_message:
            result.processing_status = "success"

        result.processing_time_ms = self._elapsed_ms(start_time)
        return result

    except Exception as e:
        return DocumentProcessingResult(
            document_id=doc_id, document_type="unknown",
```

```
            processing_status="failed",
            error_message=f"Processing error: {str(e)}",
            processing_time_ms=self._elapsed_ms(start_time)
        )

async def _process_prescription(self, result, image_bytes, ocr_result, patient_id, doc_id):
    """Process prescription — identical logic to previous version."""
    prescription = await prescription_extractor.extract(image_bytes, ocr_result)
    result.extracted_data = prescription

    for med in prescription.medications:
        confidence = ConfidenceCalculator.calculate_medication_confidence(
            source_type="prescription_photo",
            ocr_avg_confidence=prescription.ocr_avg_confidence,
            drug_match_score=med.drug_match_score,
            dosage_parsed=med.dosage is not None,
            date_found=prescription.prescription_date is not None,
            patient_confirmed=False
        )

        if confidence.final_score < 0.30:
            result.confirmation_needed.append({
                "type": "medication_unclear",
                "original_text": med.original_text,
                "message": f"We couldn't read this medication clearly: '{med.original_text}'."
            })
            continue

        fhir_resource = FHIRConverter.medication_to_fhir(
            medication=med, patient_id=patient_id,
            source_document_id=doc_id, confidence=confidence,
            prescription_date=prescription.prescription_date,
            doctor_name=prescription.doctor_name
        )

        node = await phig_builder.add_node(
            patient_id=patient_id, node_type="medication",
            display_name=f"{med.verified_name or med.name} {med.dosage or ''}".strip(),
            fhir_resource=fhir_resource,
            clinical_code=med.rxnorm_code,
            coding_system="rxnorm" if med.rxnorm_code else None,
            confidence_score=confidence.final_score,
            confidence_source="prescription_photo",
            confidence_details=confidence.__dict__,
            source_document_id=doc_id,
            effective_start=prescription.prescription_date
        )
        result.nodes_created.append(node)
```

```
            if med.needs_patient_confirmation:
                options = [m["name"] for m in med.suggested_matches[:3]] if
med.suggested_matches else []
                result.confirmation_needed.append({
                    "type": "medication_ambiguous",
                    "node_id": node["id"],
                    "original_text": med.original_text,
                    "best_guess": med.verified_name,
                    "options": options,
                    "message": f"Is '{med.original_text}' → {med.verified_name}?"
                })

        # Add diagnosis and provider nodes (same logic as before)
        if prescription.diagnosis:
            diag_fhir = FHIRConverter.diagnosis_to_fhir(
                diagnosis_text=prescription.diagnosis,
                patient_id=patient_id,
                confidence_score=min(0.75, prescription.ocr_avg_confidence),
                source="prescription_photo"
            )
            diag_node = await phig_builder.add_node(
                patient_id=patient_id, node_type="condition",
                display_name=prescription.diagnosis,
                fhir_resource=diag_fhir,
                confidence_score=min(0.75, prescription.ocr_avg_confidence),
                confidence_source="prescription_photo",
                source_document_id=doc_id
            )
            result.nodes_created.append(diag_node)

        if prescription.doctor_name:
            provider_node = await phig_builder.add_or_find_provider(
                patient_id=patient_id,
                doctor_name=prescription.doctor_name,
                speciality=prescription.doctor_speciality
            )
            for med_node in result.nodes_created:
                if med_node.get("node_type") == "medication":
                    edge = await phig_builder.add_edge(
                        patient_id=patient_id,
                        source_node_id=med_node["id"],
                        target_node_id=provider_node["id"],
                        edge_type="prescribed_by",
                        confidence_score=min(med_node.get("confidence_score", 0.5), 0.85)
                    )
                    result.edges_created.append(edge)
```

```
    async def _process_lab_report(self, result, image_bytes, ocr_result, patient_id, doc_id):
        """Process lab report — identical logic to previous version."""
        lab_report = await lab_report_extractor.extract(image_bytes, ocr_result)
        result.extracted_data = lab_report

        for test in lab_report.tests:
            confidence = ConfidenceCalculator.calculate_lab_confidence(
                source_type="lab_report_photo",
                ocr_avg_confidence=lab_report.ocr_avg_confidence,
                value_parsed=test.value is not None,
                unit_recognized=test.unit is not None,
                reference_range_found=(test.reference_range_low is not None
                                       or test.reference_range_high is not None),
                patient_confirmed=False
            )
            fhir_resource = FHIRConverter.lab_test_to_fhir(
                test=test, patient_id=patient_id,
                source_document_id=doc_id, confidence=confidence,
                report_date=lab_report.report_date, lab_name=lab_report.lab_name
            )
            node = await phig_builder.add_node(
                patient_id=patient_id, node_type="lab_value",
                display_name=f"{test.test_name}: {test.value} {test.unit or ''}".strip(),
                fhir_resource=fhir_resource,
                clinical_code=test.loinc_code,
                coding_system="loinc" if test.loinc_code else None,
                confidence_score=confidence.final_score,
                confidence_source="lab_report_photo",
                source_document_id=doc_id,
                effective_start=lab_report.report_date
            )
            result.nodes_created.append(node)

    async def _process_medicine_strip(self, result, image_bytes, ocr_result, patient_id,
doc_id):
        """Process medicine strip — identical logic to previous version."""
        from pipeline.prescription_extractor import ExtractedMedication
        strip = await medicine_strip_reader.extract(image_bytes, ocr_result)
        result.extracted_data = strip

        if strip.brand_name or strip.generic_name:
            confidence = ConfidenceCalculator.calculate_medication_confidence(
                source_type="medicine_strip_photo",
                ocr_avg_confidence=strip.ocr_avg_confidence,
                drug_match_score=strip.drug_match_score,
                dosage_parsed=strip.dosage_from_packaging is not None,
                date_found=False,
                patient_confirmed=False
```

```
            )
            med = ExtractedMedication(
                original_text=f"{strip.brand_name} {strip.dosage_from_packaging or ''}".strip(),
                name=strip.generic_name or strip.brand_name or "",
                verified_name=strip.generic_name,
                rxnorm_code=strip.rxnorm_code,
                dosage=strip.dosage_from_packaging,
                drug_match_score=strip.drug_match_score,
                ocr_confidence=strip.ocr_avg_confidence
            )
            fhir_resource = FHIRConverter.medication_to_fhir(
                medication=med, patient_id=patient_id,
                source_document_id=doc_id, confidence=confidence
            )
            node = await phig_builder.add_node(
                patient_id=patient_id, node_type="medication",
                display_name=f"{strip.generic_name or strip.brand_name}
{strip.dosage_from_packaging or ''}".strip(),
                fhir_resource=fhir_resource,
                clinical_code=strip.rxnorm_code,
                coding_system="rxnorm" if strip.rxnorm_code else None,
                confidence_score=confidence.final_score,
                confidence_source="medicine_strip_photo",
                source_document_id=doc_id
            )
            result.nodes_created.append(node)
            result.confirmation_needed.append({
                "type": "frequency_needed",
                "node_id": node["id"],
                "medication_name": strip.generic_name or strip.brand_name,
                "message": f"How many times a day do you take {strip.generic_name or
strip.brand_name}?"
            })

    def _elapsed_ms(self, start: datetime) -> int:
        return int((datetime.utcnow() - start).total_seconds() * 1000)


document_pipeline = DocumentPipeline()
```

PART 3: PHIG BUILDER, AGENTS & API ROUTES

---

8. PHIG GRAPH BUILDER

Identical to previous Part 3 — phig_builder.py with add_node, add_edge, check_interactions_for_node (with severity modifiers), evaluate_care_gaps (PostgreSQL + Azure AI Search RAG), get_medication_subgraph, get_full_patient_graph, and all helper methods.

See previous version for complete code. No changes needed.

---

9. MULTI-AGENT SYSTEM

Medication Agent, Care Gap Agent, History Agent, and Orchestrator are identical to previous Part 3.

Only change in Orchestrator: WhatsApp channel references removed.

9.1 Orchestrator (Revised — Web Only)

```
# agents/orchestrator.py

from agents.medication_agent import medication_agent
from agents.care_gap_agent import care_gap_agent
from agents.history_agent import history_agent
from agents.base_agent import AgentResponse
from services.azure_openai import openai_service
from services.azure_translator import translator_service
from dataclasses import dataclass
from typing import Optional
import asyncio


@dataclass
class OrchestratorResponse:
    message: str
    language: str
    agents_used: list
    alerts: list
    care_gaps: list
    recommendations: list
    confidence: float


SYNTHESIS_PROMPT = """You are CareOrbit's response synthesizer.
Multiple specialist agents analyzed a patient's query.
Combine their findings into ONE coherent response.
```

```
Rules:
- Lead with the most clinically important finding
- Use simple language (8th grade reading level)
- If any alerts exist, mention them prominently
- End with clear, actionable next steps
- Be warm and supportive, not clinical or scary
- For interactions: explain the risk without causing panic
- Always recommend discussing with their doctor
- Keep response concise and clear

Agent responses:
{agent_context}

Respond with the synthesized message for the patient."""


class Orchestrator:
    """Routes queries to relevant agents and synthesizes responses."""

    MEDICATION_KEYWORDS = [
        "medicine", "medication", "drug", "tablet", "capsule", "dawai",
        "interaction", "side effect", "dosage", "prescription", "dawa"
    ]
    CARE_GAP_KEYWORDS = [
        "checkup", "screening", "test", "due", "overdue", "exam",
        "check-up", "jaanch"
    ]
    HISTORY_KEYWORDS = [
        "summary", "history", "overview", "complete", "picture",
        "everything", "all", "sab", "poora"
    ]

    async def process_query(self, patient_id: str, message: str,
                            language: str = "en") -> OrchestratorResponse:

        # 1. Detect language if auto
        if language == "auto":
            detected = await translator_service.detect_language(message)
            language = detected["language"] if detected["confidence"] > 0.7 else "en"

        # 2. Translate to English for agent processing
        english_message = message
        if language != "en":
            english_message = await translator_service.translate(message, "en", language)

        # 3. Route to agents
        agents_to_run = self._route_agents(english_message)
```

```
    # 4. Run agents in parallel
    tasks = [agent.analyze(patient_id, english_message) for agent in agents_to_run]
    agent_responses: list[AgentResponse] = await asyncio.gather(*tasks)

    # 5. Synthesize
    agent_context = "\n\n".join([
        f"=== {r.agent_name} (confidence: {r.confidence:.2f}) ===\n{r.response}"
        for r in agent_responses
    ])

    synthesized = await openai_service.chat(
        system_prompt=SYNTHESIS_PROMPT.format(agent_context=agent_context),
        user_message=english_message,
        temperature=0.5
    )

    # 6. Translate response if needed
    final_message = synthesized
    if language != "en":
        final_message = await translator_service.translate(synthesized, language)

    # 7. Collect alerts and recommendations
    all_alerts = []
    all_care_gaps = []
    all_recommendations = []
    for r in agent_responses:
        all_alerts.extend(r.alerts)
        all_care_gaps.extend(r.care_gaps)
        all_recommendations.extend(r.recommendations)

    avg_confidence = (
        sum(r.confidence for r in agent_responses) / len(agent_responses)
        if agent_responses else 0.0
    )

    return OrchestratorResponse(
        message=final_message, language=language,
        agents_used=[r.agent_name for r in agent_responses],
        alerts=all_alerts, care_gaps=all_care_gaps,
        recommendations=all_recommendations,
        confidence=avg_confidence
    )

def _route_agents(self, message: str) -> list:
    msg_lower = message.lower()
    agents = []
    if any(kw in msg_lower for kw in self.MEDICATION_KEYWORDS):
        agents.append(medication_agent)
```

```
        if any(kw in msg_lower for kw in self.CARE_GAP_KEYWORDS):
            agents.append(care_gap_agent)
        if any(kw in msg_lower for kw in self.HISTORY_KEYWORDS):
            agents.append(history_agent)
        if not agents:
            agents = [medication_agent, care_gap_agent, history_agent]
        return agents

orchestrator = Orchestrator()
```

---

10. API ROUTES (All with RBAC + Audit)

10.1 Document Upload

```
# api/routes/documents.py

from fastapi import APIRouter, UploadFile, File, Depends, HTTPException, Request, Query
from pipeline.document_pipeline import document_pipeline
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.audit import log_audit
from database.connection import async_session
from sqlalchemy import text

router = APIRouter()


@router.post("/upload")
async def upload_document(
    request: Request,
    file: UploadFile = File(...),
    patient_id: str = Query(None),
    current_user=Depends(get_current_user)
):
    """Upload a medical document with RBAC and audit."""
    target_patient = patient_id or current_user["id"]

    # RBAC: Verify caller can edit this patient's data
    await verify_patient_access(current_user["id"], target_patient, required_permission="edit")

    # Validate file
    allowed_types = ["image/jpeg", "image/png", "image/webp", "image/heic"]
    if file.content_type not in allowed_types:
        raise HTTPException(400, f"File type {file.content_type} not supported.")
    if file.size and file.size > 10 * 1024 * 1024:
```

```
        raise HTTPException(400, "File too large. Maximum 10MB.")

    image_bytes = await file.read()
    ext = file.filename.rsplit(".", 1)[-1] if file.filename else "jpg"

    # Get patient email for notification
    async with async_session() as session:
        result = await session.execute(
            text("SELECT email, name FROM users WHERE id = :pid"),
            {"pid": target_patient}
        )
        patient_info = result.mappings().first()

    result = await document_pipeline.process_document(
        image_bytes=image_bytes,
        patient_id=target_patient,
        uploaded_by=current_user["id"],
        patient_email=patient_info["email"] if patient_info else None,
        patient_name=patient_info["name"] if patient_info else None,
        file_extension=ext
    )

    return {
        "document_id": result.document_id,
        "document_type": result.document_type,
        "status": result.processing_status,
        "nodes_created": len(result.nodes_created),
        "interaction_alerts": result.interaction_alerts,
        "care_gap_alerts": result.care_gap_alerts,
        "confirmation_needed": result.confirmation_needed,
        "processing_time_ms": result.processing_time_ms,
        "error": result.error_message
    }
```

10.2 Chat

```
# api/routes/chat.py

from fastapi import APIRouter, Depends, Request
from pydantic import BaseModel
from agents.orchestrator import orchestrator
from api.middleware.auth import get_current_user
from api.middleware.audit import log_audit

router = APIRouter()

class ChatRequest(BaseModel):
```

```
    message: str
    patient_id: str = None   # For caregiver querying on behalf of patient
    language: str = "auto"


@router.post("/query")
async def chat_query(req: ChatRequest, request: Request,
                     current_user=Depends(get_current_user)):
    target_patient = req.patient_id or current_user["id"]

    # RBAC: Caregivers need at least 'view' permission to query
    if target_patient != current_user["id"]:
        from api.middleware.rbac import verify_patient_access
        await verify_patient_access(current_user["id"], target_patient, "view")

    response = await orchestrator.process_query(
        patient_id=target_patient,
        message=req.message,
        language=req.language
    )

    # Audit
    await log_audit(
        user_id=current_user["id"], patient_id=target_patient,
        action="CHAT_QUERY", request=request,
        metadata={"agents_used": response.agents_used, "language": response.language}
    )

    return {
        "message": response.message,
        "language": response.language,
        "agents_used": response.agents_used,
        "alerts": response.alerts,
        "care_gaps": response.care_gaps,
        "recommendations": response.recommendations,
        "confidence": response.confidence
    }
```

10.3 Patient Profile & Overview

```
# api/routes/patients.py

from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.audit import log_audit
```

```
from graph.phig_builder import phig_builder
from database.connection import async_session
from sqlalchemy import text
from typing import Optional

router = APIRouter()


class UpdateProfileRequest(BaseModel):
    date_of_birth: Optional[str] = None
    gender: Optional[str] = None
    blood_group: Optional[str] = None
    height_cm: Optional[float] = None
    weight_kg: Optional[float] = None
    city: Optional[str] = None
    state: Optional[str] = None


@router.get("/overview")
async def get_overview(request: Request, patient_id: str = None,
                       current_user=Depends(get_current_user)):
    """Get patient health overview (dashboard data)."""
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    graph = await phig_builder.get_full_patient_graph(target)

    await log_audit(current_user["id"], target, "VIEW_OVERVIEW", request=request)

    return graph["summary"]


@router.get("/medications")
async def get_medications(request: Request, patient_id: str = None,
                          current_user=Depends(get_current_user)):
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    subgraph = await phig_builder.get_medication_subgraph(target)

    await log_audit(current_user["id"], target, "VIEW_MEDICATIONS", request=request)

    return subgraph


@router.get("/care-gaps")
async def get_care_gaps(request: Request, patient_id: str = None,
                        current_user=Depends(get_current_user)):
```

```
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    care_gaps = await phig_builder.evaluate_care_gaps(target)

    await log_audit(current_user["id"], target, "VIEW_CARE_GAPS", request=request)

    return care_gaps


@router.get("/alerts")
async def get_alerts(request: Request, patient_id: str = None,
                     current_user=Depends(get_current_user)):
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    # Get active interaction alerts from PHIG edges
    async with async_session() as session:
        result = await session.execute(text("""
            SELECT e.properties, e.reasoning_chain, e.confidence_score,
                   s.display_name as drug_a, t.display_name as drug_b
            FROM phig_edges e
            JOIN phig_nodes s ON s.id = e.source_node_id
            JOIN phig_nodes t ON t.id = e.target_node_id
            WHERE e.patient_id = :pid AND e.edge_type = 'interacts_with' AND e.is_active =
TRUE
        """), {"pid": target})
        interactions = result.mappings().all()

    alerts = []
    for i in interactions:
        import json
        props = i["properties"]
        if isinstance(props, str):
            props = json.loads(props)
        alerts.append({
            "type": "drug_interaction",
            "severity": props.get("severity", "Unknown"),
            "drug_a": i["drug_a"],
            "drug_b": i["drug_b"],
            "description": props.get("description", ""),
            "clinical_action": props.get("clinical_action", ""),
            "confidence": float(i["confidence_score"]) if i["confidence_score"] else 0.5
        })

    return alerts
```

```
@router.put("/profile")
async def update_profile(req: UpdateProfileRequest, request: Request,
                         patient_id: str = None,
                         current_user=Depends(get_current_user)):
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "edit")

    updates = {k: v for k, v in req.dict().items() if v is not None}
    if not updates:
        raise HTTPException(400, "No fields to update")

    # Build dynamic SET clause
    set_parts = []
    params = {"pid": target}
    for key, value in updates.items():
        set_parts.append(f"{key} = :{key}")
        params[key] = value
    set_parts.append("updated_at = NOW()")

    async with async_session() as session:
        await session.execute(text(
            f"UPDATE patient_profiles SET {', '.join(set_parts)} WHERE user_id = :pid"
        ), params)
        await session.commit()

    await log_audit(current_user["id"], target, "UPDATE_PROFILE", request=request,
                    metadata={"fields_updated": list(updates.keys())})

    return {"status": "updated", "fields": list(updates.keys())}
```

10.4 Health Summary

```
# api/routes/summary.py

from fastapi import APIRouter, Depends, Request
from fastapi.responses import StreamingResponse
from graph.phig_builder import phig_builder
from services.azure_blob import blob_service
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.audit import log_audit
from utils.summary_generator import generate_health_summary_pdf
from uuid import uuid4
import io

router = APIRouter()
```

```
@router.get("/generate")
async def generate_summary(request: Request, patient_id: str = None,
                           language: str = "en",
                           current_user=Depends(get_current_user)):
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    graph = await phig_builder.get_full_patient_graph(target)
    if graph["summary"]["total_nodes"] == 0:
        return {"error": "No health data available. Upload prescriptions first."}

    pdf_bytes = await generate_health_summary_pdf(target, graph)

    summary_id = str(uuid4())
    blob_url = await blob_service.upload_health_summary_pdf(
        pdf_bytes, target, summary_id
    )

    # Audit
    await log_audit(
        current_user["id"], target, "GENERATE_SUMMARY",
        resource_type="summary", resource_id=summary_id,
        request=request
    )

    return StreamingResponse(
        io.BytesIO(pdf_bytes),
        media_type="application/pdf",
        headers={
            "Content-Disposition": f"attachment;
filename=CareOrbit_Summary_{summary_id[:8]}.pdf"
        }
    )
```

10.5 Caregiver Management

```
# api/routes/caregivers.py

from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel, EmailStr
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.audit import log_audit
from database.connection import async_session
from sqlalchemy import text
```

```
router = APIRouter()


class AddCaregiverRequest(BaseModel):
    caregiver_email: EmailStr
    relationship: str                       # son, daughter, spouse, parent, sibling, other
    permission_level: str = "view"              # view, edit, full


class UpdateCaregiverRequest(BaseModel):
    permission_level: str


@router.post("/add")
async def add_caregiver(req: AddCaregiverRequest, request: Request,
                        current_user=Depends(get_current_user)):
    """Patient adds a caregiver to their account."""
    patient_id = current_user["id"]

    if req.relationship not in ("son", "daughter", "spouse", "parent", "sibling", "other"):
        raise HTTPException(400, "Invalid relationship type")
    if req.permission_level not in ("view", "edit", "full"):
        raise HTTPException(400, "Invalid permission level")

    # Find caregiver by email
    async with async_session() as session:
        result = await session.execute(
            text("SELECT id, name FROM users WHERE email = :email AND is_active =
TRUE"),
            {"email": req.caregiver_email}
        )
        caregiver = result.mappings().first()

    if not caregiver:
        raise HTTPException(404, "No registered user found with that email. "
                            "They need to create a CareOrbit account first.")

    caregiver_id = str(caregiver["id"])
    if caregiver_id == patient_id:
        raise HTTPException(400, "You cannot add yourself as a caregiver")

    async with async_session() as session:
        await session.execute(text("""
            INSERT INTO caregiver_links (caregiver_user_id, patient_user_id, relationship,
permission_level)
            VALUES (:cid, :pid, :rel, :perm)
            ON CONFLICT (caregiver_user_id, patient_user_id)
            DO UPDATE SET relationship = :rel, permission_level = :perm,
```

```
                    is_active = TRUE, revoked_at = NULL
            """), {"cid": caregiver_id, "pid": patient_id,
                   "rel": req.relationship, "perm": req.permission_level})
            await session.commit()

        await log_audit(patient_id, patient_id, "ADD_CAREGIVER", request=request,
                        metadata={"caregiver_id": caregiver_id, "permission": req.permission_level})

        return {"status": "added", "caregiver_name": caregiver["name"],
                "permission_level": req.permission_level}


@router.delete("/{caregiver_id}")
async def revoke_caregiver(caregiver_id: str, request: Request,
                           current_user=Depends(get_current_user)):
    """Patient revokes a caregiver's access."""
    patient_id = current_user["id"]

    async with async_session() as session:
        await session.execute(text("""
            UPDATE caregiver_links
            SET is_active = FALSE, revoked_at = NOW()
            WHERE caregiver_user_id = :cid AND patient_user_id = :pid
        """), {"cid": caregiver_id, "pid": patient_id})
        await session.commit()

    await log_audit(patient_id, patient_id, "REVOKE_CAREGIVER", request=request,
                    metadata={"revoked_caregiver": caregiver_id})

    return {"status": "revoked"}


@router.get("/my-patients")
async def my_patients(current_user=Depends(get_current_user)):
    """Caregiver views all patients they have access to."""
    async with async_session() as session:
        result = await session.execute(text("""
            SELECT cl.patient_user_id, cl.relationship, cl.permission_level,
                   u.name AS patient_name
            FROM caregiver_links cl
            JOIN users u ON u.id = cl.patient_user_id
            WHERE cl.caregiver_user_id = :uid AND cl.is_active = TRUE
        """), {"uid": current_user["id"]})
        patients = result.mappings().all()

    return [dict(p) for p in patients]
```

```
@router.get("/my-caregivers")
async def my_caregivers(current_user=Depends(get_current_user)):
    """Patient views all caregivers with access to their data."""
    async with async_session() as session:
        result = await session.execute(text("""
            SELECT cl.caregiver_user_id, cl.relationship, cl.permission_level,
                   u.name AS caregiver_name, u.email AS caregiver_email
            FROM caregiver_links cl
            JOIN users u ON u.id = cl.caregiver_user_id
            WHERE cl.patient_user_id = :uid AND cl.is_active = TRUE
        """), {"uid": current_user["id"]})
        caregivers = result.mappings().all()

    return [dict(c) for c in caregivers]
```

10.6 Email Reminders

```
# api/routes/reminders.py

from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.audit import log_audit
from database.connection import async_session
from sqlalchemy import text
from typing import Optional
from uuid import uuid4

router = APIRouter()


class CreateReminderRequest(BaseModel):
    medication_node_id: str
    reminder_time: str          # "08:00", "14:00", "21:00"
    days_of_week: list[int] = [1, 2, 3, 4, 5, 6, 7]


@router.post("/create")
async def create_reminder(req: CreateReminderRequest, request: Request,
                          patient_id: str = None,
                          current_user=Depends(get_current_user)):
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "edit")

    # Verify medication node exists and belongs to patient
    async with async_session() as session:
```

```
        node = await session.execute(text("""
            SELECT id, display_name FROM phig_nodes
            WHERE id = :nid AND patient_id = :pid AND node_type = 'medication' AND is_active
= TRUE
        """), {"nid": req.medication_node_id, "pid": target})
        node = node.mappings().first()

    if not node:
        raise HTTPException(404, "Medication not found in patient's records")

    reminder_id = str(uuid4())
    async with async_session() as session:
        await session.execute(text("""
            INSERT INTO medication_reminders (id, patient_id, medication_node_id,
                                              reminder_time, days_of_week, delivery_method)
            VALUES (:id, :pid, :nid, :time, :days, 'email')
        """), {
            "id": reminder_id, "pid": target,
            "nid": req.medication_node_id,
            "time": req.reminder_time,
            "days": req.days_of_week
        })
        await session.commit()

    return {"reminder_id": reminder_id, "status": "created",
            "medication": node["display_name"], "time": req.reminder_time}


@router.get("/list")
async def list_reminders(patient_id: str = None,
                         current_user=Depends(get_current_user)):
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    async with async_session() as session:
        result = await session.execute(text("""
            SELECT r.id, r.reminder_time, r.days_of_week, r.is_active,
                   r.last_sent_at, r.last_confirmed_at,
                   n.display_name AS medication_name
            FROM medication_reminders r
            JOIN phig_nodes n ON n.id = r.medication_node_id
            WHERE r.patient_id = :pid AND r.is_active = TRUE
            ORDER BY r.reminder_time
        """), {"pid": target})
        reminders = result.mappings().all()

    return [dict(r) for r in reminders]
```

```
@router.delete("/{reminder_id}")
async def delete_reminder(reminder_id: str, request: Request,
                          current_user=Depends(get_current_user)):
    async with async_session() as session:
        # Verify ownership
        result = await session.execute(text("""
            SELECT patient_id FROM medication_reminders WHERE id = :rid
        """), {"rid": reminder_id})
        reminder = result.mappings().first()

    if not reminder:
        raise HTTPException(404, "Reminder not found")

    await verify_patient_access(current_user["id"], str(reminder["patient_id"]), "edit")

    async with async_session() as session:
        await session.execute(text("""
            UPDATE medication_reminders SET is_active = FALSE WHERE id = :rid
        """), {"rid": reminder_id})
        await session.commit()

    return {"status": "deleted"}
```

10.7 Data Confirmation (New — Replaces WhatsApp Buttons)

```
# api/routes/confirmations.py

from fastapi import APIRouter, Depends, Request
from pydantic import BaseModel
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.audit import log_audit
from graph.phig_builder import phig_builder
from database.connection import async_session
from sqlalchemy import text
from typing import Optional

router = APIRouter()


class ConfirmMedicationRequest(BaseModel):
    node_id: str
    confirmed: bool                  # True = correct, False = wrong
    corrected_name: Optional[str] = None
    frequency: Optional[str] = None    # For medicine strips
```

```
@router.post("/confirm")
async def confirm_extracted_data(req: ConfirmMedicationRequest, request: Request,
                                 current_user=Depends(get_current_user)):
    """Patient confirms or corrects AI-extracted data via web UI."""
    # Get node to verify ownership
    node = await phig_builder._get_node(req.node_id)
    if not node:
        return {"error": "Node not found"}

    patient_id = str(node["patient_id"])
    await verify_patient_access(current_user["id"], patient_id, "edit")

    if req.confirmed:
        # Boost confidence — patient says extraction is correct
        await phig_builder._update_node_confidence(
            req.node_id, score=0.85,
            source="patient_confirmed",
            details={"patient_confirmed": True, "confirmed_by": current_user["id"]}
        )
        await log_audit(current_user["id"], patient_id, "CONFIRM_DATA",
                        resource_type="phig_node", resource_id=req.node_id,
                        request=request, metadata={"confirmed": True})

        return {"status": "confirmed", "new_confidence": 0.85}

    else:
        if req.corrected_name:
            # Patient corrected the medication name — update node
            from utils.drug_database import drug_db
            match = await drug_db.fuzzy_match(req.corrected_name)
            if match.confidence > 0.8:
                # Update node with corrected name
                async with async_session() as session:
                    await session.execute(text("""
                        UPDATE phig_nodes
                        SET display_name = :name,
                            clinical_code = :code,
                            coding_system = CASE WHEN :code IS NOT NULL THEN 'rxnorm' ELSE
NULL END,
                            confidence_score = 0.85,
                            confidence_source = 'patient_corrected',
                            updated_at = NOW()
                        WHERE id = :nid
                    """), {
                        "name": f"{match.generic_name} {node.get('display_name', '').split()[-1] if ' ' in
node.get('display_name', '') else ''}".strip(),
                        "code": match.rxnorm_code,
```

```
                "nid": req.node_id
            })
            await session.commit()

        return {"status": "corrected", "new_name": match.generic_name}

    # Patient says it's wrong but no correction — deactivate the node
    async with async_session() as session:
        await session.execute(text("""
            UPDATE phig_nodes SET is_active = FALSE, updated_at = NOW()
            WHERE id = :nid
        """), {"nid": req.node_id})
        await session.commit()

    return {"status": "removed"}
```

PART 4: MONETIZATION LAYER, FRONTEND, TESTING & DEPLOYMENT

---

11. SUBSCRIPTION & TIER SYSTEM

11.1 Schema Additions for Monetization

```
-- ═══════════════════════════════════════════════════════════
-- SUBSCRIPTION & TIER MANAGEMENT
-- ═══════════════════════════════════════════════════════════

-- User subscription tier
CREATE TYPE subscription_tier AS ENUM ('free', 'premium_individual', 'premium_family');
CREATE TYPE subscription_status AS ENUM ('active', 'trial', 'past_due', 'cancelled',
'expired');

CREATE TABLE subscriptions (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    owner_user_id     UUID NOT NULL REFERENCES users(id),   -- Person who pays
    tier              subscription_tier NOT NULL DEFAULT 'free',
    status            subscription_status NOT NULL DEFAULT 'active',
    -- Pricing
    price_cents       INTEGER DEFAULT 0,              -- 799 = $7.99, 1999 = $19.99
    currency          VARCHAR(3) DEFAULT 'USD',
    billing_cycle     VARCHAR(10) DEFAULT 'monthly',  -- monthly, yearly
    -- Dates
```

```
    started_at          TIMESTAMPTZ DEFAULT NOW(),
    current_period_start TIMESTAMPTZ DEFAULT NOW(),
    current_period_end  TIMESTAMPTZ,
    trial_end           TIMESTAMPTZ,
    cancelled_at        TIMESTAMPTZ,
    -- Payment provider (Stripe/Razorpay for India)
    payment_provider    VARCHAR(20),                -- stripe, razorpay
    external_sub_id     VARCHAR(255),               -- Stripe/Razorpay subscription ID
    -- Family plan: which users are covered
    family_member_ids   UUID[] DEFAULT '{}',        -- Up to 3 for family plan
    max_family_members  INTEGER DEFAULT 1,          -- 1 for individual, 3 for family
    created_at          TIMESTAMPTZ DEFAULT NOW(),
    updated_at          TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_subscriptions_owner ON subscriptions(owner_user_id, status);
CREATE INDEX idx_subscriptions_tier ON subscriptions(tier, status) WHERE status =
'active';

-- Usage tracking (for enforcing free tier limits)
CREATE TABLE usage_tracking (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id             UUID NOT NULL REFERENCES users(id),
    metric              VARCHAR(50) NOT NULL,
    -- Metrics tracked:
    -- 'documents_uploaded_month' — 100 limit for free
    -- 'ai_queries_month' — unlimited for all (but batch for free)
    -- 'summaries_generated_month' — 5 for free, unlimited premium
    current_value       INTEGER DEFAULT 0,
    max_allowed         INTEGER,                    -- NULL = unlimited
    period_start        TIMESTAMPTZ DEFAULT DATE_TRUNC('month', NOW()),
    period_end          TIMESTAMPTZ DEFAULT DATE_TRUNC('month', NOW()) + INTERVAL
'1 month',
    UNIQUE(user_id, metric, period_start)
);
CREATE INDEX idx_usage_user ON usage_tracking(user_id, metric, period_start);

-- Ad impression tracking (for revenue reporting + frequency capping)
CREATE TABLE ad_impressions (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id             UUID NOT NULL REFERENCES users(id),
    ad_slot             VARCHAR(30) NOT NULL,       -- dashboard_banner, feed_native,
sidebar
    ad_provider         VARCHAR(30) DEFAULT 'internal',  -- internal, google_adsense,
pharma_sponsor
    campaign_id         VARCHAR(100),
    impression_type     VARCHAR(20) DEFAULT 'view',     -- view, click, dismiss
    created_at          TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Partitioned by month in production for query performance
CREATE INDEX idx_ad_impressions_user ON ad_impressions(user_id, created_at DESC);
CREATE INDEX idx_ad_impressions_campaign ON ad_impressions(campaign_id,
created_at DESC);

-- Add tier column to users table
ALTER TABLE users ADD COLUMN subscription_tier subscription_tier DEFAULT 'free';
```

11.2 Tier Configuration (Feature Gates)

```
# utils/tier_config.py

"""
Central configuration for what each tier can access.
Every feature gate checks this config — no hardcoded limits elsewhere.
"""

TIER_LIMITS = {
    "free": {
        # Document limits
        "documents_per_month": 100,
        "data_retention_months": 24,          # 2-year rolling window
        "max_document_size_mb": 10,

        # AI features
        "ai_processing_priority": "standard",  # standard = queued if busy
        "drug_interaction_db_size": "basic",    # 10K drugs
        "predictive_analytics": False,
        "ai_query_delay_seconds": 5,           # Slight delay for free users

        # Summaries
        "summaries_per_month": 5,
        "custom_reports": False,

        # Family
        "max_caregivers": 2,
        "family_dashboard": False,

        # Integrations
        "wearable_sync": False,
        "ehr_integration": False,
        "export_formats": ["pdf"],

        # Support
        "support_priority": "standard",
        "video_support": False,
```

```
    # Ads
    "shows_ads": True,
    "ad_slots": ["dashboard_banner", "feed_native_1", "feed_native_2", "sidebar"],
    "max_ads_per_session": 4,

    # Storage
    "max_storage_mb": 500,
},

"premium_individual": {
    "documents_per_month": None,          # Unlimited
    "data_retention_months": None,        # Lifetime
    "max_document_size_mb": 25,

    "ai_processing_priority": "high",     # Immediate processing
    "drug_interaction_db_size": "comprehensive",  # 50K+ drugs
    "predictive_analytics": True,
    "ai_query_delay_seconds": 0,

    "summaries_per_month": None,
    "custom_reports": True,

    "max_caregivers": 5,
    "family_dashboard": False,

    "wearable_sync": True,
    "ehr_integration": True,
    "export_formats": ["pdf", "fhir_json", "csv", "apple_health"],

    "support_priority": "high",
    "video_support": True,

    "shows_ads": False,
    "ad_slots": [],
    "max_ads_per_session": 0,

    "max_storage_mb": None,               # Unlimited
},

"premium_family": {
    # Everything from premium_individual, plus:
    "documents_per_month": None,
    "data_retention_months": None,
    "max_document_size_mb": 25,

    "ai_processing_priority": "high",
    "drug_interaction_db_size": "comprehensive",
    "predictive_analytics": True,
```

```
        "ai_query_delay_seconds": 0,

        "summaries_per_month": None,
        "custom_reports": True,

        "max_caregivers": 10,
        "family_dashboard": True,            # Unified family view
        "max_family_members": 3,
        "cross_member_alerts": True,         # "Dad's med interacts with Mom's condition"
        "elderly_care_monitoring": True,

        "wearable_sync": True,
        "ehr_integration": True,
        "export_formats": ["pdf", "fhir_json", "csv", "apple_health"],

        "support_priority": "high",
        "video_support": True,

        "shows_ads": False,
        "ad_slots": [],
        "max_ads_per_session": 0,

        "max_storage_mb": None,
    }
}


def get_tier_limits(tier: str) -> dict:
    return TIER_LIMITS.get(tier, TIER_LIMITS["free"])


def check_feature_allowed(tier: str, feature: str) -> bool:
    limits = get_tier_limits(tier)
    return limits.get(feature, False)


def get_limit(tier: str, metric: str):
    limits = get_tier_limits(tier)
    return limits.get(metric)
```

11.3 Feature Gate Middleware

```
# api/middleware/feature_gate.py

from fastapi import HTTPException
from database.connection import async_session
from sqlalchemy import text
```

```
from utils.tier_config import get_tier_limits, get_limit


async def get_user_tier(user_id: str) -> str:
    """Get user's subscription tier."""
    async with async_session() as session:
        result = await session.execute(
            text("SELECT subscription_tier FROM users WHERE id = :uid"),
            {"uid": user_id}
        )
        row = result.first()
    return row[0] if row else "free"


async def check_usage_limit(user_id: str, metric: str) -> dict:
    """
    Check if user has exceeded their tier's usage limit.
    Returns: {"allowed": bool, "current": int, "limit": int|None, "tier": str}
    """
    tier = await get_user_tier(user_id)
    limit = get_limit(tier, metric)

    if limit is None:
        return {"allowed": True, "current": 0, "limit": None, "tier": tier}

    async with async_session() as session:
        result = await session.execute(text("""
            SELECT current_value FROM usage_tracking
            WHERE user_id = :uid AND metric = :metric
              AND period_start = DATE_TRUNC('month', NOW())
        """), {"uid": user_id, "metric": metric})
        row = result.first()

    current = row[0] if row else 0
    return {
        "allowed": current < limit,
        "current": current,
        "limit": limit,
        "tier": tier
    }


async def increment_usage(user_id: str, metric: str):
    """Increment usage counter for a metric."""
    tier = await get_user_tier(user_id)
    limit = get_limit(tier, metric)

    async with async_session() as session:
```

```
        await session.execute(text("""
            INSERT INTO usage_tracking (user_id, metric, current_value, max_allowed,
                                        period_start, period_end)
            VALUES (:uid, :metric, 1, :limit,
                    DATE_TRUNC('month', NOW()),
                    DATE_TRUNC('month', NOW()) + INTERVAL '1 month')
            ON CONFLICT (user_id, metric, period_start)
            DO UPDATE SET current_value = usage_tracking.current_value + 1,
                          max_allowed = :limit
        """), {"uid": user_id, "metric": metric, "limit": limit})
        await session.commit()


async def enforce_document_limit(user_id: str):
    """Check document upload limit before processing."""
    usage = await check_usage_limit(user_id, "documents_per_month")
    if not usage["allowed"]:
        raise HTTPException(
            status_code=429,
            detail={
                "error": "Monthly document upload limit reached",
                "current": usage["current"],
                "limit": usage["limit"],
                "tier": usage["tier"],
                "upgrade_message": "Upgrade to Premium for unlimited uploads.",
                "upgrade_url": "/settings/subscription"
            }
        )


async def enforce_summary_limit(user_id: str):
    """Check summary generation limit."""
    usage = await check_usage_limit(user_id, "summaries_per_month")
    if not usage["allowed"]:
        raise HTTPException(
            status_code=429,
            detail={
                "error": "Monthly summary generation limit reached",
                "current": usage["current"],
                "limit": usage["limit"],
                "tier": usage["tier"],
                "upgrade_message": "Upgrade to Premium for unlimited health summaries."
            }
        )


async def get_processing_priority(user_id: str) -> str:
    """Determine processing priority based on tier."""
```

```
    tier = await get_user_tier(user_id)
    limits = get_tier_limits(tier)
    return limits.get("ai_processing_priority", "standard")
```

11.4 Updated Document Upload Route (with Feature Gates)

```
# api/routes/documents.py — Updated with tier enforcement

from fastapi import APIRouter, UploadFile, File, Depends, HTTPException, Request, Query
from pipeline.document_pipeline import document_pipeline
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.feature_gate import enforce_document_limit, increment_usage,
get_processing_priority
from database.connection import async_session
from sqlalchemy import text
import asyncio

router = APIRouter()


@router.post("/upload")
async def upload_document(
    request: Request,
    file: UploadFile = File(...),
    patient_id: str = Query(None),
    current_user=Depends(get_current_user)
):
    target_patient = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target_patient, required_permission="edit")

    # Feature gate: check document limit for uploader's tier
    await enforce_document_limit(current_user["id"])

    # Validate file
    allowed_types = ["image/jpeg", "image/png", "image/webp", "image/heic"]
    if file.content_type not in allowed_types:
        raise HTTPException(400, f"File type {file.content_type} not supported.")
    if file.size and file.size > 10 * 1024 * 1024:
        raise HTTPException(400, "File too large. Maximum 10MB.")

    image_bytes = await file.read()
    ext = file.filename.rsplit(".", 1)[-1] if file.filename else "jpg"

    # Check processing priority
    priority = await get_processing_priority(current_user["id"])
```

```
    # Free tier: add slight delay during high load (not punitive, just prioritization)
    if priority == "standard":
        # In production, this would be a queue position check
        # For MVP, a simple 2-second yield to let premium users go first
        await asyncio.sleep(2)

    # Get patient info for email notification
    async with async_session() as session:
        result = await session.execute(
            text("SELECT email, name FROM users WHERE id = :pid"),
            {"pid": target_patient}
        )
        patient_info = result.mappings().first()

    result = await document_pipeline.process_document(
        image_bytes=image_bytes,
        patient_id=target_patient,
        uploaded_by=current_user["id"],
        patient_email=patient_info["email"] if patient_info else None,
        patient_name=patient_info["name"] if patient_info else None,
        file_extension=ext
    )

    # Increment usage counter
    await increment_usage(current_user["id"], "documents_per_month")

    return {
        "document_id": result.document_id,
        "document_type": result.document_type,
        "status": result.processing_status,
        "nodes_created": len(result.nodes_created),
        "interaction_alerts": result.interaction_alerts,
        "care_gap_alerts": result.care_gap_alerts,
        "confirmation_needed": result.confirmation_needed,
        "processing_time_ms": result.processing_time_ms,
        "error": result.error_message
    }
```

11.5 Subscription API Routes

```
# api/routes/subscriptions.py

from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel
from api.middleware.auth import get_current_user
from api.middleware.audit import log_audit
from api.middleware.feature_gate import check_usage_limit
```

```
from utils.tier_config import TIER_LIMITS
from database.connection import async_session
from sqlalchemy import text
from datetime import datetime, timedelta
from uuid import uuid4

router = APIRouter()


class UpgradeRequest(BaseModel):
    tier: str              # premium_individual, premium_family
    billing_cycle: str = "monthly"


@router.get("/current")
async def get_subscription(current_user=Depends(get_current_user)):
    """Get current subscription details + usage."""
    async with async_session() as session:
        result = await session.execute(text("""
            SELECT s.tier, s.status, s.price_cents, s.billing_cycle,
                   s.current_period_end, s.family_member_ids, s.max_family_members
            FROM subscriptions s
            WHERE s.owner_user_id = :uid AND s.status IN ('active', 'trial')
            ORDER BY s.created_at DESC LIMIT 1
        """), {"uid": current_user["id"]})
        sub = result.mappings().first()

    tier = sub["tier"] if sub else "free"
    limits = TIER_LIMITS[tier]

    # Get current usage
    docs_usage = await check_usage_limit(current_user["id"], "documents_per_month")
    summary_usage = await check_usage_limit(current_user["id"], "summaries_per_month")

    return {
        "tier": tier,
        "status": sub["status"] if sub else "active",
        "billing_cycle": sub["billing_cycle"] if sub else None,
        "current_period_end": sub["current_period_end"] if sub else None,
        "family_members": sub["family_member_ids"] if sub else [],
        "max_family_members": sub["max_family_members"] if sub else 1,
        "usage": {
            "documents": {
                "used": docs_usage["current"],
                "limit": docs_usage["limit"],
                "unlimited": docs_usage["limit"] is None
            },
            "summaries": {
```

```
            "used": summary_usage["current"],
            "limit": summary_usage["limit"],
            "unlimited": summary_usage["limit"] is None
        }
    },
    "features": {
        "shows_ads": limits["shows_ads"],
        "ai_priority": limits["ai_processing_priority"],
        "predictive_analytics": limits["predictive_analytics"],
        "wearable_sync": limits["wearable_sync"],
        "custom_reports": limits["custom_reports"],
        "family_dashboard": limits["family_dashboard"],
    }
}


@router.get("/plans")
async def get_available_plans():
    """Return available subscription plans for upgrade UI."""
    return {
        "plans": [
            {
                "tier": "free",
                "name": "Free — Complete Care Coordination",
                "price_monthly_cents": 0,
                "price_yearly_cents": 0,
                "highlights": [
                    "Unlimited conditions & medications tracking",
                    "Real-time drug interaction monitoring",
                    "AI-powered care coordination insights",
                    "Health timeline and trend analysis",
                    "100 document uploads per month",
                    "5 health summaries per month",
                    "Ad-supported"
                ]
            },
            {
                "tier": "premium_individual",
                "name": "Premium Individual",
                "price_monthly_cents": 799,
                "price_yearly_cents": 7990,   # ~$66.58/year = 2 months free
                "highlights": [
                    "Everything in Free",
                    "Zero advertisements",
                    "Priority AI processing (2x faster)",
                    "Unlimited document uploads & summaries",
                    "Advanced predictive analytics",
                    "Wearable device sync",
```

```
                "Custom health reports",
                "Lifetime data retention",
                "Priority support"
            ]
        },
        {
            "tier": "premium_family",
            "name": "Family Premium (up to 3 people)",
            "price_monthly_cents": 1999,
            "price_yearly_cents": 19990,
            "highlights": [
                "Everything in Premium for ALL 3 members",
                "Unified family health dashboard",
                "Cross-member health alerts",
                "Elderly care monitoring",
                "Multi-caregiver coordination hub",
                "Family health calendar",
                "Emergency medical packet",
                "Effective $6.66/person/month"
            ]
        }
    ]
}


@router.post("/upgrade")
async def upgrade_subscription(req: UpgradeRequest, request: Request,
                               current_user=Depends(get_current_user)):
    """
    Initiate subscription upgrade.
    For MVP: This creates the subscription record directly.
    For production: This would create a Stripe/Razorpay checkout session.
    """
    if req.tier not in ("premium_individual", "premium_family"):
        raise HTTPException(400, "Invalid tier")

    price_map = {
        "premium_individual": {"monthly": 799, "yearly": 7990},
        "premium_family": {"monthly": 1999, "yearly": 19990},
    }
    price = price_map[req.tier][req.billing_cycle]
    max_family = 3 if req.tier == "premium_family" else 1

    period_end = datetime.utcnow() + (
        timedelta(days=30) if req.billing_cycle == "monthly" else timedelta(days=365)
    )

    sub_id = str(uuid4())
```

```
    async with async_session() as session:
        # Deactivate any existing subscription
        await session.execute(text("""
            UPDATE subscriptions SET status = 'cancelled', cancelled_at = NOW()
            WHERE owner_user_id = :uid AND status = 'active'
        """), {"uid": current_user["id"]})

        # Create new subscription
        await session.execute(text("""
            INSERT INTO subscriptions (id, owner_user_id, tier, status,
                                       price_cents, billing_cycle,
                                       current_period_end, max_family_members)
            VALUES (:id, :uid, :tier, 'active',
                    :price, :cycle, :period_end, :max_fam)
        """), {
            "id": sub_id, "uid": current_user["id"],
            "tier": req.tier, "price": price,
            "cycle": req.billing_cycle,
            "period_end": period_end, "max_fam": max_family
        })

        # Update user tier
        await session.execute(text("""
            UPDATE users SET subscription_tier = :tier WHERE id = :uid
        """), {"tier": req.tier, "uid": current_user["id"]})

        await session.commit()

    await log_audit(current_user["id"], current_user["id"], "UPGRADE_SUBSCRIPTION",
                    request=request, metadata={"tier": req.tier, "price_cents": price})

    # In production: return Stripe/Razorpay checkout URL
    return {
        "subscription_id": sub_id,
        "tier": req.tier,
        "status": "active",
        "period_end": period_end.isoformat(),
        # "checkout_url": "https://checkout.stripe.com/..."  # Production
    }
```

---

12. FRONTEND (Next.js 14) — With Monetization

12.1 API Client with Token Refresh

```
// src/lib/api.ts
```

```
import axios, { AxiosError } from 'axios';

const API_BASE = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000';

const api = axios.create({
  baseURL: `${API_BASE}/api`,
  timeout: 60000,
  headers: { 'Content-Type': 'application/json' }
});

let isRefreshing = false;
let failedQueue: Array<{ resolve: Function; reject: Function }> = [];

function processQueue(error: any, token: string | null) {
  failedQueue.forEach(({ resolve, reject }) => {
    error ? reject(error) : resolve(token);
  });
  failedQueue = [];
}

// Attach access token
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('careorbit_access_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Handle 401 with automatic token refresh
api.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as any;

    if (error.response?.status === 401 && !originalRequest._retry) {
      if (isRefreshing) {
        return new Promise((resolve, reject) => {
          failedQueue.push({ resolve, reject });
        }).then((token) => {
          originalRequest.headers.Authorization = `Bearer ${token}`;
          return api(originalRequest);
        });
      }

      originalRequest._retry = true;
      isRefreshing = true;

      try {
```

```
        const refreshToken = localStorage.getItem('careorbit_refresh_token');
        if (!refreshToken) throw new Error('No refresh token');

        const { data } = await axios.post(`${API_BASE}/api/auth/refresh`, {
          refresh_token: refreshToken
        });

        localStorage.setItem('careorbit_access_token', data.access_token);
        localStorage.setItem('careorbit_refresh_token', data.refresh_token);

        processQueue(null, data.access_token);
        originalRequest.headers.Authorization = `Bearer ${data.access_token}`;
        return api(originalRequest);
      } catch (refreshError) {
        processQueue(refreshError, null);
        localStorage.removeItem('careorbit_access_token');
        localStorage.removeItem('careorbit_refresh_token');
        window.location.href = '/login';
        return Promise.reject(refreshError);
      } finally {
        isRefreshing = false;
      }
    }

    return Promise.reject(error);
  }
);

export default api;
```

12.2 Subscription Store

```
// src/stores/useSubscriptionStore.ts

import { create } from 'zustand';
import api from '@/lib/api';

interface SubscriptionState {
  tier: 'free' | 'premium_individual' | 'premium_family';
  showsAds: boolean;
  usage: {
    documents: { used: number; limit: number | null };
    summaries: { used: number; limit: number | null };
  };
  features: Record<string, boolean>;
  loading: boolean;
  fetchSubscription: () => Promise<void>;
```

```
}

export const useSubscriptionStore = create<SubscriptionState>((set) => ({
  tier: 'free',
  showsAds: true,
  usage: {
    documents: { used: 0, limit: 100 },
    summaries: { used: 0, limit: 5 },
  },
  features: {},
  loading: true,
  fetchSubscription: async () => {
    try {
      const { data } = await api.get('/subscriptions/current');
      set({
        tier: data.tier,
        showsAds: data.features.shows_ads,
        usage: data.usage,
        features: data.features,
        loading: false,
      });
    } catch {
      set({ loading: false });
    }
  },
}));
```

12.3 Ad Slot Component (Free Tier)

```
// src/components/ads/AdSlot.tsx

'use client';
import { useSubscriptionStore } from '@/stores/useSubscriptionStore';

interface AdSlotProps {
  slot: 'dashboard_banner' | 'feed_native' | 'sidebar';
  className?: string;
}

export default function AdSlot({ slot, className = '' }: AdSlotProps) {
  const { showsAds, tier } = useSubscriptionStore();

  // Premium users never see ads
  if (!showsAds) return null;

  // MVP: Show placeholder ads that demonstrate the concept
  // Production: This would integrate Google AdSense or custom ad server
```

```
  const adContent = {
    dashboard_banner: {
      title: 'Managing Diabetes?',
      body: 'Learn about continuous glucose monitoring from AccuCheck.',
      cta: 'Learn More',
      label: 'Sponsored Health Insight',
    },
    feed_native: {
      title: 'Heart Health Awareness',
      body: 'The AHA recommends regular blood pressure monitoring for adults over 40.',
      cta: 'Read Guidelines',
      label: 'Health Education Partner',
    },
    sidebar: {
      title: 'Apollo Pharmacy',
      body: 'Get your medications delivered. 15% off first order.',
      cta: 'Order Now',
      label: 'Sponsored',
    },
  };

  const ad = adContent[slot] || adContent.feed_native;

  return (
    <div className={`bg-gray-50 border border-gray-200 rounded-lg p-4 ${className}`}>
      <p className="text-xs text-gray-400 mb-2">{ad.label}</p>
      <p className="font-medium text-gray-700 text-sm">{ad.title}</p>
      <p className="text-gray-500 text-sm mt-1">{ad.body}</p>
      <div className="flex items-center justify-between mt-3">
        <button className="text-sm text-blue-600 font-medium hover:underline">
          {ad.cta}
        </button>
        <button className="text-xs text-gray-400 hover:text-gray-600">✕</button>
      </div>
      {/* Upgrade nudge */}
      <p className="text-xs text-gray-400 mt-2 border-t pt-2">
        <a href="/settings/subscription" className="text-blue-500 hover:underline">
          Upgrade to Premium
        </a> for an ad-free experience
      </p>
    </div>
  );
}
```

12.4 Usage Limit Banner

```
// src/components/ui/UsageBanner.tsx
```

```
'use client';
import { useSubscriptionStore } from '@/stores/useSubscriptionStore';
import { AlertTriangle } from 'lucide-react';

export default function UsageBanner() {
  const { tier, usage } = useSubscriptionStore();
  if (tier !== 'free') return null;

  const docsUsed = usage.documents.used;
  const docsLimit = usage.documents.limit || 100;
  const percentage = (docsUsed / docsLimit) * 100;

  if (percentage < 70) return null;

  const isExhausted = percentage >= 100;

  return (
    <div className={`rounded-lg p-3 mb-4 flex items-center gap-3 ${
      isExhausted ? 'bg-red-50 border border-red-200' : 'bg-yellow-50 border
border-yellow-200'
    }`}>
      <AlertTriangle className={`w-5 h-5 flex-shrink-0 ${
        isExhausted ? 'text-red-500' : 'text-yellow-600'
      }`} />
      <div className="flex-1">
        <p className={`text-sm font-medium ${isExhausted ? 'text-red-800' :
'text-yellow-800'}`}>
          {isExhausted
            ? `Document upload limit reached (${docsUsed}/${docsLimit} this month)`
            : `${docsUsed} of ${docsLimit} documents used this month`
          }
        </p>
      </div>
      <a href="/settings/subscription"
         className="text-sm font-medium text-blue-600 hover:underline whitespace-nowrap">
        Upgrade →
      </a>
    </div>
  );
}
```

12.5 Dashboard Home (with Ads + Usage)

```
// src/app/(dashboard)/page.tsx

'use client';
```

```
import { useEffect, useState } from 'react';
import { Heart, Pill, TestTube, AlertTriangle, Users, Activity } from 'lucide-react';
import { useSubscriptionStore } from '@/stores/useSubscriptionStore';
import api from '@/lib/api';
import AdSlot from '@/components/ads/AdSlot';
import UsageBanner from '@/components/ui/UsageBanner';

export default function DashboardHome() {
  const [overview, setOverview] = useState<any>(null);
  const [alerts, setAlerts] = useState<any[]>([]);
  const [gaps, setGaps] = useState<any[]>([]);
  const { fetchSubscription, tier } = useSubscriptionStore();

  useEffect(() => {
    fetchSubscription();
    Promise.all([
      api.get('/patients/overview'),
      api.get('/patients/alerts'),
      api.get('/patients/care-gaps'),
    ]).then(([o, a, g]) => {
      setOverview(o.data);
      setAlerts(a.data);
      setGaps(g.data);
    });
  }, []);

  if (!overview) return <div className="p-8 text-center text-gray-500">Loading...</div>;

  const cards = [
    { label: 'Conditions', value: overview.conditions, icon: Heart, color: 'text-red-500' },
    { label: 'Medications', value: overview.medications, icon: Pill, color: 'text-blue-500' },
    { label: 'Lab Results', value: overview.lab_values, icon: TestTube, color: 'text-green-500' },
    { label: 'Care Gaps', value: overview.care_gaps, icon: AlertTriangle, color:
'text-orange-500' },
    { label: 'Providers', value: overview.providers, icon: Users, color: 'text-purple-500' },
    { label: 'Interactions', value: overview.interactions, icon: Activity, color: 'text-red-600' },
  ];

  return (
    <div className="p-6 space-y-6 max-w-5xl mx-auto">
      <div className="flex items-center justify-between">
        <h1 className="text-2xl font-bold text-gray-800">Your Health Dashboard</h1>
        {tier !== 'free' && (
          <span className="px-3 py-1 bg-blue-100 text-blue-700 text-xs font-medium
rounded-full">
            ✨ Premium
          </span>
        )}
```

```
</div>

<UsageBanner />

{/* Urgent alerts — always at top, all tiers */}
{alerts.length > 0 && (
  <div className="bg-red-50 border border-red-200 rounded-xl p-4 space-y-3">
    <h2 className="text-lg font-semibold text-red-800 flex items-center gap-2">
      <AlertTriangle className="w-5 h-5" /> Drug Interactions Detected
    </h2>
    {alerts.map((alert: any, i: number) => (
      <div key={i} className="bg-white rounded-lg p-3 border border-red-100">
        <p className="font-medium text-red-700">
          {alert.drug_a} + {alert.drug_b} — {alert.severity}
        </p>
        <p className="text-sm text-gray-600 mt-1">{alert.description}</p>
        <p className="text-sm text-blue-600 mt-1 italic">{alert.clinical_action}</p>
      </div>
    ))}
  </div>
)}

{/* Overview cards */}
<div className="grid grid-cols-2 md:grid-cols-3 gap-4">
  {cards.map(card => (
    <div key={card.label} className="bg-white rounded-xl shadow-sm border p-4">
      <div className="flex items-center gap-3">
        <card.icon className={`w-8 h-8 ${card.color}`} />
        <div>
          <p className="text-2xl font-bold">{card.value}</p>
          <p className="text-sm text-gray-500">{card.label}</p>
        </div>
      </div>
    </div>
  ))}
</div>

{/* Ad slot — only shows for free tier */}
<AdSlot slot="dashboard_banner" />

{/* Care gaps */}
{gaps.length > 0 && (
  <div className="bg-orange-50 border border-orange-200 rounded-xl p-4 space-y-3">
    <h2 className="text-lg font-semibold text-orange-800">Overdue Screenings</h2>
    {gaps.map((gap: any, i: number) => (
      <div key={i} className="bg-white rounded-lg p-3 border border-orange-100">
        <p className="font-medium">{gap.screening_name}</p>
        <p className="text-sm text-gray-500">
```

```
              {gap.guideline_source} — {gap.frequency}
              {gap.overdue_by_months && ` — ${gap.overdue_by_months} months overdue`}
            </p>
          </div>
        ))}
      </div>
    ))}

    {/* Ad slot — native in feed */}
    <AdSlot slot="feed_native" />
  </div>
 );
}
```

12.6 Pricing Page Component

```
// src/app/pricing/page.tsx

'use client';
import { useState, useEffect } from 'react';
import { Check } from 'lucide-react';
import api from '@/lib/api';

export default function PricingPage() {
 const [plans, setPlans] = useState<any[]>([]);
 const [billing, setBilling] = useState<'monthly' | 'yearly'>('monthly');

 useEffect(() => {
  api.get('/subscriptions/plans').then(res => setPlans(res.data.plans));
 }, []);

 const getPrice = (plan: any) => {
  if (plan.tier === 'free') return 'Free';
  const cents = billing === 'monthly' ? plan.price_monthly_cents : plan.price_yearly_cents;
  const monthly = billing === 'yearly' ? (cents / 12 / 100).toFixed(2) : (cents /
100).toFixed(2);
  return `$${monthly}/mo`;
 };

 return (
  <div className="max-w-4xl mx-auto p-8">
   <h1 className="text-3xl font-bold text-center text-gray-800">
    Care Coordination for Everyone
   </h1>
   <p className="text-center text-gray-500 mt-2">
    Full features free. Premium for power users.
   </p>
```

```
    {/* Billing toggle */}
    <div className="flex justify-center mt-6 gap-3">
      <button onClick={() => setBilling('monthly')}
            className={`px-4 py-2 rounded-lg text-sm font-medium transition-colors
                  ${billing === 'monthly' ? 'bg-blue-600 text-white' : 'bg-gray-100
text-gray-600'}`}>
        Monthly
      </button>
      <button onClick={() => setBilling('yearly')}
            className={`px-4 py-2 rounded-lg text-sm font-medium transition-colors
                  ${billing === 'yearly' ? 'bg-blue-600 text-white' : 'bg-gray-100
text-gray-600'}`}>
        Yearly (2 months free)
      </button>
    </div>

    {/* Plan cards */}
    <div className="grid md:grid-cols-3 gap-6 mt-8">
      {plans.map(plan => (
        <div key={plan.tier}
            className={`rounded-xl border p-6 ${
              plan.tier === 'premium_individual'
                ? 'border-blue-500 ring-2 ring-blue-100 relative'
                : 'border-gray-200'
            }`}>
          {plan.tier === 'premium_individual' && (
            <span className="absolute -top-3 left-1/2 -translate-x-1/2 bg-blue-600
                  text-white text-xs px-3 py-1 rounded-full">
              Most Popular
            </span>
          )}
          <h3 className="text-lg font-bold text-gray-800">{plan.name}</h3>
          <p className="text-3xl font-bold mt-3 text-gray-900">{getPrice(plan)}</p>
          {billing === 'yearly' && plan.tier !== 'free' && (
            <p className="text-sm text-green-600 mt-1">Save 17% with annual</p>
          )}
          <ul className="mt-6 space-y-3">
            {plan.highlights.map((h: string, i: number) => (
              <li key={i} className="flex items-start gap-2 text-sm text-gray-600">
                <Check className="w-4 h-4 text-green-500 flex-shrink-0 mt-0.5" />
                <span>{h}</span>
              </li>
            ))}
          </ul>
          <button className={`w-full mt-6 py-2 rounded-lg font-medium transition-colors ${
            plan.tier === 'free'
              ? 'bg-gray-100 text-gray-700 hover:bg-gray-200'
```

```
              : 'bg-blue-600 text-white hover:bg-blue-700'
          }`}>
            {plan.tier === 'free' ? 'Current Plan' : 'Upgrade'}
          </button>
        </div>
      ))}
    </div>
  </div>
 );
}
```

---

13. TESTING STRATEGY

13.1 Feature Gate Tests

```
# tests/test_feature_gates.py

import pytest
from utils.tier_config import TIER_LIMITS, get_tier_limits, get_limit


class TestTierConfig:
    """Verify tier configuration is consistent and correct."""

    def test_free_tier_has_document_limit(self):
        limit = get_limit("free", "documents_per_month")
        assert limit == 100

    def test_premium_individual_unlimited_documents(self):
        limit = get_limit("premium_individual", "documents_per_month")
        assert limit is None

    def test_free_tier_shows_ads(self):
        limits = get_tier_limits("free")
        assert limits["shows_ads"] is True

    def test_premium_no_ads(self):
        for tier in ("premium_individual", "premium_family"):
            limits = get_tier_limits(tier)
            assert limits["shows_ads"] is False

    def test_free_tier_standard_priority(self):
        limits = get_tier_limits("free")
        assert limits["ai_processing_priority"] == "standard"
```

```
    def test_premium_high_priority(self):
        for tier in ("premium_individual", "premium_family"):
            limits = get_tier_limits(tier)
            assert limits["ai_processing_priority"] == "high"

    def test_family_plan_allows_3_members(self):
        limits = get_tier_limits("premium_family")
        assert limits["max_family_members"] == 3
        assert limits["family_dashboard"] is True

    def test_individual_no_family_dashboard(self):
        limits = get_tier_limits("premium_individual")
        assert limits["family_dashboard"] is False

    def test_free_tier_summary_limit(self):
        limit = get_limit("free", "summaries_per_month")
        assert limit == 5

    def test_premium_unlimited_summaries(self):
        limit = get_limit("premium_individual", "summaries_per_month")
        assert limit is None

    def test_all_tiers_have_all_keys(self):
        """Every tier must define every feature key."""
        free_keys = set(TIER_LIMITS["free"].keys())
        for tier_name, tier_config in TIER_LIMITS.items():
            tier_keys = set(tier_config.keys())
            missing = free_keys - tier_keys
            assert not missing, f"Tier '{tier_name}' missing keys: {missing}"

    def test_unknown_tier_returns_free(self):
        limits = get_tier_limits("nonexistent_tier")
        assert limits == TIER_LIMITS["free"]
```

13.2 Auth & Security Tests

```
# tests/test_auth.py

import pytest
from api.middleware.auth import (
    hash_password, verify_password,
    create_access_token
)
from jose import jwt
from config import get_settings
```

```
settings = get_settings()


class TestPasswordHashing:
    def test_hash_is_not_plaintext(self):
        hashed = hash_password("mypassword123")
        assert hashed != "mypassword123"
        assert hashed.startswith("$2b$")

    def test_verify_correct_password(self):
        hashed = hash_password("secure_pass_456")
        assert verify_password("secure_pass_456", hashed) is True

    def test_reject_wrong_password(self):
        hashed = hash_password("correct_password")
        assert verify_password("wrong_password", hashed) is False


class TestAccessTokens:
    def test_token_contains_user_id(self):
        token = create_access_token("user-123-uuid")
        payload = jwt.decode(token, settings.JWT_SECRET,
                             algorithms=[settings.JWT_ALGORITHM])
        assert payload["sub"] == "user-123-uuid"
        assert payload["type"] == "access"

    def test_token_has_expiry(self):
        token = create_access_token("user-123-uuid")
        payload = jwt.decode(token, settings.JWT_SECRET,
                             algorithms=[settings.JWT_ALGORITHM])
        assert "exp" in payload

    def test_invalid_secret_rejects(self):
        token = create_access_token("user-123-uuid")
        with pytest.raises(Exception):
            jwt.decode(token, "wrong-secret",
                       algorithms=[settings.JWT_ALGORITHM])
```

13.3 Confidence Calculator + Drug Database Tests

(Identical to previous Part 4 — test_confidence.py and test_drug_database.py. No changes.)

---

14. DEMO DATA (Revised with Tier Info)

```
# database/seed_demo.py
```

```
"""
Synthetic demo patients for Imagine Cup demonstration.
Now includes subscription tier data.
"""

DEMO_PATIENTS = [
    {
        "name": "Ramesh Kumar",
        "email": "ramesh.demo@careorbit.dev",
        "phone": "+919876543210",
        "tier": "free",                    # Shows ads, has limits
        "dob": "1958-03-15",
        "gender": "male",
        "blood_group": "B+",
        "city": "Durgapur",
        "state": "West Bengal",
        "language": "hi",
        "conditions": [
            {"name": "Type 2 Diabetes Mellitus", "code": "E11.9", "confidence": 0.85},
            {"name": "Essential Hypertension", "code": "I10", "confidence": 0.82},
            {"name": "Dyslipidemia", "code": "E78.5", "confidence": 0.78},
        ],
        "medications": [
            {"name": "Metformin 500mg", "generic": "Metformin", "rxnorm": "6809",
             "dosage": "500mg BD after food", "confidence": 0.85,
             "source": "prescription_photo", "doctor": "Dr. Amit Roy"},
            {"name": "Amlodipine 5mg", "generic": "Amlodipine", "rxnorm": "17767",
             "dosage": "5mg OD morning", "confidence": 0.82,
             "source": "medicine_strip_photo", "doctor": "Dr. Amit Roy"},
            {"name": "Atorvastatin 10mg", "generic": "Atorvastatin", "rxnorm": "83367",
             "dosage": "10mg HS", "confidence": 0.78,
             "source": "prescription_photo", "doctor": "Dr. S. Das"},
            {"name": "Ecosprin 75mg", "generic": "Aspirin", "rxnorm": "1191",
             "dosage": "75mg OD after lunch", "confidence": 0.90,
             "source": "medicine_strip_photo", "doctor": "Dr. S. Das"},
            {"name": "Ibuprofen 400mg", "generic": "Ibuprofen", "rxnorm": "5640",
             "dosage": "400mg SOS for knee pain", "confidence": 0.65,
             "source": "patient_text_input", "doctor": "Unknown"},
        ],
        "labs": [
            {"name": "HbA1c", "value": "7.8", "unit": "%", "ref_low": "4.0", "ref_high": "5.6",
             "abnormal": True, "loinc": "4548-4", "date": "2025-11-20", "confidence": 0.88},
            {"name": "Serum Creatinine", "value": "1.4", "unit": "mg/dL",
             "ref_low": "0.7", "ref_high": "1.3",
             "abnormal": True, "loinc": "2160-0", "date": "2025-11-20", "confidence": 0.87},
            {"name": "eGFR", "value": "52", "unit": "mL/min/1.73m2",
             "ref_low": "90", "ref_high": "120",
```

```
            "abnormal": True, "loinc": "33914-3", "date": "2025-11-20", "confidence": 0.86},
           {"name": "Total Cholesterol", "value": "218", "unit": "mg/dL",
            "ref_low": "0", "ref_high": "200",
            "abnormal": True, "loinc": "2093-3", "date": "2025-10-05", "confidence": 0.85},
           {"name": "Fasting Blood Glucose", "value": "156", "unit": "mg/dL",
            "ref_low": "70", "ref_high": "100",
            "abnormal": True, "loinc": "1558-6", "date": "2025-12-10", "confidence": 0.88},
        ],
        "demo_scenario": {
           "interaction": "Metformin + Ibuprofen — severity ELEVATED due to creatinine 1.4,
eGFR 52, age 68",
           "care_gaps": ["Diabetic retinopathy screening (never done)",
                         "Comprehensive foot exam (never done)",
                         "Nephrology referral (eGFR declining)"],
           "demo_value": "Shows multi-hop PHIG reasoning and free tier with ads"
        }
    },
    {
        "name": "Priya Sharma",
        "email": "priya.demo@careorbit.dev",
        "phone": "+919876543211",
        "tier": "premium_individual",          # No ads, priority processing
        "dob": "1972-08-22",
        "gender": "female",
        "blood_group": "O+",
        "city": "Bangalore",
        "state": "Karnataka",
        "language": "en",
        "conditions": [
           {"name": "Hypothyroidism", "code": "E03.9", "confidence": 0.90},
           {"name": "Essential Hypertension", "code": "I10", "confidence": 0.80},
        ],
        "medications": [
           {"name": "Thyronorm 50mcg", "generic": "Levothyroxine", "rxnorm": "10582",
            "dosage": "50mcg OD empty stomach", "confidence": 0.90,
            "source": "medicine_strip_photo", "doctor": "Dr. Meena Gupta"},
           {"name": "Telma 40mg", "generic": "Telmisartan", "rxnorm": "73494",
            "dosage": "40mg OD morning", "confidence": 0.82,
            "source": "prescription_photo", "doctor": "Dr. R. Krishnan"},
        ],
        "labs": [
           {"name": "TSH", "value": "6.8", "unit": "mIU/L",
            "ref_low": "0.4", "ref_high": "4.0",
            "abnormal": True, "loinc": "3016-3", "date": "2025-09-15", "confidence": 0.88},
        ],
        "demo_scenario": {
           "interaction": "None — clean medication list",
           "care_gaps": ["TSH recheck overdue (5+ months, was abnormal)"],
```

```
            "demo_value": "Shows premium tier — no ads, fast processing, clean UI"
        }
    },
    {
        "name": "The Gupta Family",
        "email": "anand.demo@careorbit.dev",
        "phone": "+919876543212",
        "tier": "premium_family",              # Family dashboard demo
        "members": [
            {
                "name": "Anand Gupta (Father, 72)",
                "conditions": ["Type 2 Diabetes", "Chronic Kidney Disease Stage 3a"],
                "medications": ["Metformin 1000mg", "Insulin Glargine", "Losartan 50mg"],
            },
            {
                "name": "Sunita Gupta (Mother, 68)",
                "conditions": ["Rheumatoid Arthritis", "Osteoporosis"],
                "medications": ["Methotrexate 15mg/week", "Calcium + Vit D", "Fosamax
70mg/week"],
            },
            {
                "name": "Vikram Gupta (Son, 38, Primary Caregiver)",
                "conditions": [],
                "medications": [],
            }
        ],
        "demo_scenario": {
            "cross_member_alert": "Father's declining kidney function means Metformin dose
needs review — son gets alert",
            "family_dashboard": "Vikram sees both parents' health status in one view",
            "care_coordination": "Mother's rheumatology and father's nephrology appointments
auto-coordinated",
            "demo_value": "Shows family plan value — unified dashboard, cross-member alerts,
caregiver coordination"
        }
    }
]
```

---

15. REVISED 12-WEEK BUILD TIMELINE

```
WEEK 1-2: Foundation + Security
├── Day 1-2: Azure provisioning (all 9 services)
├── Day 3-4: PostgreSQL schema (full schema including subscriptions)
├── Day 5-6: FastAPI skeleton + healthcare-grade auth
│            (access tokens, refresh tokens, token rotation)
├── Day 7-8: RBAC middleware + audit trail
```

├── Day 9-10: Azure service clients (OpenAI, Vision, Blob, Search, Translator)
└── Deliverable: Authenticated API with security layers passing tests

WEEK 3-4: Photo-to-FHIR Pipeline
├── Day 1-2: Document classifier + Azure Vision OCR
├── Day 3-5: Prescription extractor + drug database (fuzzy matching)
├── Day 6-7: Lab report extractor
├── Day 8: Medicine strip reader
├── Day 9-10: FHIR converter + confidence scoring
└── Deliverable: Upload prescription photo → get structured FHIR data with confidence

WEEK 5-6: PHIG + Agents
├── Day 1-3: PHIG builder (add_node, add_edge, check_interactions, evaluate_care_gaps)
├── Day 4-5: Medication Agent with severity modifier logic
├── Day 6-7: Care Gap Agent with Azure AI Search RAG
├── Day 8-9: Orchestrator (multi-agent routing + synthesis + translation)
├── Day 10: Master document pipeline (photo → PHIG → agents → alerts)
└── Deliverable: Full pipeline works end-to-end with drug interaction detection

WEEK 7-8: Monetization + Email
├── Day 1-2: Subscription system (tiers, feature gates, usage tracking)
├── Day 3-4: Tier enforcement middleware (document limits, processing priority)
├── Day 5-6: Email service (reminders, interaction alerts, upload results)
├── Day 7-8: Subscription API routes (current plan, available plans, upgrade)
├── Day 9-10: Seed demo data (3 patient scenarios across 3 tiers)
└── Deliverable: Complete tiered system with email notifications

WEEK 9-10: Frontend
├── Day 1-2: Next.js setup + auth flow (register, login, token refresh)
├── Day 3-4: Dashboard home + ad slots + usage banners
├── Day 5-6: Document upload + confirmation flow + real-time processing feedback
├── Day 7-8: AI chat interface + care gaps page + medications page
├── Day 9-10: Health summary generation + pricing page + subscription management
└── Deliverable: Complete web UI with tier-aware features

WEEK 11-12: Polish + Demo
├── Day 1-2: End-to-end testing (all 3 demo scenarios)
├── Day 3-4: Fix bugs, optimize processing time, handle edge cases
├── Day 5-6: Confirmations page UI, Hindi translation verification
├── Day 7-8: Demo script rehearsal, screen recordings for submission
├── Day 9-10: Final submission materials, pitch deck alignment
└── Deliverable: Imagine Cup submission ready

---

16. COMPLETE PROJECT FILE MAP (Revised)

careorbit-backend/

```
├── main.py
├── config.py
├── requirements.txt
├── Dockerfile
├── .env.example
│
├── database/
│   ├── connection.py
│   ├── schema.sql                # Full schema with subscriptions + usage
│   ├── seed_guidelines.json
│   ├── seed_drug_interactions.json
│   └── seed_demo.py              # 3 demo patients across 3 tiers
│
├── api/
│   ├── routes/
│   │   ├── auth.py               # Register, login, refresh, logout
│   │   ├── patients.py           # Profile, overview, medications, care-gaps, alerts
│   │   ├── documents.py          # Upload with tier enforcement
│   │   ├── chat.py               # AI chat with RBAC
│   │   ├── summary.py            # PDF generation with limit check
│   │   ├── caregivers.py         # Add/revoke/list caregivers
│   │   ├── reminders.py          # Email reminder CRUD
│   │   ├── confirmations.py      # Patient data confirmation (web UI)
│   │   └── subscriptions.py      # Plans, current, upgrade
│   └── middleware/
│       ├── auth.py               # JWT + refresh tokens
│       ├── rbac.py               # Patient data access control
│       ├── audit.py              # Audit trail logging
│       └── feature_gate.py       # Tier-based feature enforcement
│
├── pipeline/
│   ├── document_classifier.py
│   ├── prescription_extractor.py
│   ├── lab_report_extractor.py
│   ├── medicine_strip_reader.py
│   ├── fhir_converter.py
│   └── document_pipeline.py      # Master pipeline with email notif
│
├── graph/
│   ├── confidence.py
│   └── phig_builder.py
│
├── agents/
│   ├── base_agent.py
│   ├── medication_agent.py
│   ├── care_gap_agent.py
│   ├── history_agent.py
│   └── orchestrator.py           # Web-only, no WhatsApp
```

```
│
├── services/
│   ├── azure_openai.py
│   ├── azure_vision.py
│   ├── azure_blob.py
│   ├── azure_search.py
│   ├── azure_translator.py
│   └── email_service.py            # Azure Communication Services
│
├── utils/
│   ├── drug_database.py
│   ├── summary_generator.py
│   ├── encryption.py               # PII field encryption
│   └── tier_config.py              # Feature gate configuration
│
└── tests/
    ├── conftest.py
    ├── test_confidence.py
    ├── test_drug_database.py
    ├── test_auth.py
    ├── test_feature_gates.py
    ├── test_pipeline_e2e.py
    └── test_agents.py


careorbit-web/
├── src/
│   ├── app/
│   │   ├── (auth)/login/page.tsx
│   │   ├── (auth)/register/page.tsx
│   │   ├── (dashboard)/page.tsx       # Dashboard with ads + usage
│   │   ├── (dashboard)/medications/
│   │   ├── (dashboard)/care-gaps/
│   │   ├── (dashboard)/upload/
│   │   ├── (dashboard)/chat/
│   │   ├── (dashboard)/summary/
│   │   ├── (dashboard)/family/        # Premium family only
│   │   ├── pricing/page.tsx           # Plan comparison
│   │   └── settings/subscription/     # Manage subscription
│   ├── components/
│   │   ├── ui/
│   │   │   ├── ConfidenceBadge.tsx
│   │   │   ├── UsageBanner.tsx
│   │   │   └── UpgradePrompt.tsx
│   │   ├── ads/
│   │   │   └── AdSlot.tsx             # Tier-aware ad placement
│   │   ├── upload/
│   │   │   ├── DocumentUploader.tsx
```

```
│   │   │   └── ConfirmationFlow.tsx
│   │   ├── chat/
│   │   │   └── ChatInterface.tsx
│   │   └── dashboard/
│   │       └── HealthOverview.tsx
│   ├── lib/
│   │   ├── api.ts                # With automatic token refresh
│   │   ├── auth.ts
│   │   └── types.ts
│   └── stores/
│       ├── usePatientStore.ts
│       └── useSubscriptionStore.ts
└── package.json
```

---

17. IMAGINE CUP DEMO SCRIPT (Revised for Business Model)

The 5-Minute Demo

MINUTE 0:00 — The Problem
"Ramesh Kumar, 68, Durgapur. Type 2 diabetes, hypertension, high cholesterol. Three doctors, none see his full picture. His son Vikram in Bangalore can't coordinate his father's care."

MINUTE 0:30 — Upload (Free Tier)
Log in as Ramesh (free tier — ads visible on dashboard).
Upload prescription photo.
[Show: Processing with AI... 20 seconds...]
Results: 3 medications extracted with confidence scores.

MINUTE 1:15 — THE WOW MOMENT: Multi-Hop Drug Interaction
"CareOrbit detected Metformin + Ibuprofen interaction. But here's what makes us different from any drug interaction checker on the market:"

Show the PHIG graph:
Metformin → interacts_with → Ibuprofen
→ severity_modified_by → Creatinine 1.4 mg/dL
→ indicates → eGFR 52 (declining kidney function)
→ patient_age → 68 (over 65)
→ SEVERITY: ELEVATED (not just 'moderate')

"No other consumer health product does multi-hop clinical reasoning across a patient's complete health graph."

MINUTE 2:15 — Care Gaps (RAG)
Show care gap detection:
"Ramesh has NEVER had a diabetic eye exam. ADA guidelines say annual screening for all diabetics. That's a care gap we found by combining his PHIG data with clinical guidelines stored in our RAG knowledge base."

MINUTE 2:45 — Health Summary PDF
Generate the one-page PDF with confidence indicators.
"This is what Ramesh takes to his next doctor visit. One page. Every medication with confidence levels. Drug interactions highlighted. Overdue screenings listed."

MINUTE 3:15 — Premium Demo (Switch to Priya)
Switch to Priya (premium individual).
"Notice: No ads. Priority processing. Clean interface. Priya's hypothyroid medication shows up instantly with 0.90 confidence from her medicine strip photo."

MINUTE 3:45 — Family Plan Demo (Switch to Gupta Family)
Switch to Vikram's family dashboard.
"Vikram manages both parents' health from Bangalore. He sees his father's declining kidney function AND his mother's upcoming rheumatology appointment — in one unified view."

Show cross-member alert:
"Father's creatinine trending up means his Metformin dose may need review. Vikram gets this alert automatically."

MINUTE 4:15 — Business Model
"CareOrbit is free for every patient. Full drug interaction monitoring, care gap detection, health summaries. Premium removes ads and adds priority processing, predictive analytics, and family coordination.

At $7.99/month individual and $19.99/month family, with 4% conversion, we're profitable by Year 4. Even worst-case, premium margins cover free user costs."

MINUTE 4:45 — Market + Innovation
"80% of global healthcare runs on paper. 5 billion people need this. We start with India's 50 million family caregivers.

Our innovation: the Confidence-Weighted Patient Health Intelligence Graph — multi-hop clinical reasoning from paper prescriptions that no one else has built."

---

18. KEY BUSINESS MODEL DECISIONS IN CODE

What the Business Model Means for Engineering Priorities

Business Decision	Technical Implementation
Free tier with full features	All core pipeline runs for all users; no feature-stripping
100 document limit (free)	usage_tracking table + enforce_document_limit middleware
5 summary limit (free)	enforce_summary_limit before PDF generation
Premium priority processing	get_processing_priority checks tier before pipeline
Ad slots in free tier	AdSlot component conditionally renders; subscription store tracks tier
Family plan (3 members)	subscriptions.family_member_ids array + family_dashboard feature flag
Cross-member alerts	phig_builder checks linked family members for interaction conflicts
2-year data retention (free)	Cron job: DELETE FROM phig_nodes WHERE patient_tier = 'free' AND created_at < NOW() - INTERVAL '2 years'
Lifetime retention (premium)	No deletion; subscription downgrade triggers retention policy
Ethical ads (health-only)	Ad content curated server-side; no programmatic ads from unknown sources in MVP

Revenue Protection in Architecture

Why free users can't game the system:

- Document limits enforced server-side (not client-side)
- Usage counters are per-user, per-month, reset automatically
- Tier stored in database, not in JWT (can't be spoofed)
- Feature gates check database on every request

Why premium is genuinely more valuable (not just "remove ads"):

- Priority processing queue position (measurable speed difference)
- Drug interaction database size (50K vs 10K compounds)
- Unlimited document history (no 2-year cutoff)
- Custom health reports (not available to free)
- Family coordination features (cross-member alerts, unified dashboard)

CareOrbit MVP V2 — Addendum: P0 Feature Additions

History Agent, Family Caregiver View, Lab Trends, Adherence Tracking

---

1. HISTORY AGENT (Complete Implementation)

```
# agents/history_agent.py

from agents.base_agent import BaseAgent, AgentResponse
from graph.phig_builder import phig_builder
from services.azure_openai import openai_service
import json


HISTORY_SYSTEM_PROMPT = """You are CareOrbit's Health History Agent.
You create clear, chronological narratives of a patient's health journey.

Your job:
1. Build a timeline from the patient's health graph data
2. Identify relationships (which doctor treats which condition)
3. Highlight trends (improving, stable, declining)
4. Note confidence levels — be transparent about data quality
5. Identify gaps in the story (missing dates, unknown prescribers)

Output JSON:
{
  "narrative": "2-3 paragraph summary of patient's health journey. Write in
               third person. Lead with current status, then chronological
               history. Mention specific doctors, dates, and confidence levels
               where relevant. Note any concerning trends.",

  "key_facts": [
    {"fact": "string", "confidence": 0.0-1.0, "source": "string"}
  ],
```

```
  "timeline": [
    {
      "date": "YYYY-MM-DD or 'Unknown'",
      "event": "description",
      "category":
"diagnosis|medication_start|medication_change|lab_result|screening|provider_visit",
      "significance": "routine|notable|concerning|critical"
    }
  ],

  "providers": [
    {
      "name": "Dr. X",
      "role": "Treats: condition A, condition B",
      "medications_prescribed": ["med1", "med2"],
      "last_seen": "date or 'Unknown'"
    }
  ],

  "health_trajectory": {
    "overall": "improving|stable|declining|mixed",
    "reasoning": "Brief explanation of trajectory assessment",
    "areas_of_concern": ["string"],
    "positive_developments": ["string"]
  },

  "data_quality": {
    "completeness": "high|moderate|low",
    "notes": "What's missing or uncertain in the data",
    "low_confidence_items": ["Items with confidence < 0.65"]
  }
}

Rules:
- Be factual and evidence-based — only state what the data supports
- Clearly distinguish between verified (🟢 ≥0.85), confirmed (🟡 ≥0.65),
  reported (🟠 ≥0.40), and uncertain (🔴 <0.40) data
- If a trend is concerning, say so directly but calmly
- Never diagnose or recommend treatment — you summarize what exists
- Use dates when available; say "date unknown" when not
- Note when multiple doctors prescribe related medications (coordination gap)
- Keep narrative accessible (8th grade reading level)
"""


class HistoryAgent(BaseAgent):
    agent_name = "History Agent"
```

```
    async def analyze(self, patient_id: str, query: str = "") -> AgentResponse:
        # Get full patient graph
        graph = await phig_builder.get_full_patient_graph(patient_id)

        if graph["summary"]["total_nodes"] == 0:
            return AgentResponse(
                agent_name=self.agent_name,
                response="No health data available yet. Upload prescriptions or lab reports to
build your health history.",
                confidence=0.0,
                reasoning_chains=[],
                alerts=[], care_gaps=[], recommendations=[]
            )

        # Format graph for LLM
        context = self._format_graph(graph)

        response_text = await openai_service.chat(
            system_prompt=HISTORY_SYSTEM_PROMPT,
            user_message=f"Patient query: {query}\n\nPatient health graph:\n{context}",
            temperature=0.3,
            response_format="json_object"
        )

        try:
            parsed = json.loads(response_text)
        except json.JSONDecodeError:
            parsed = {"narrative": response_text, "timeline": [], "providers": [],
                      "health_trajectory": {"overall": "unknown"}, "data_quality": {"completeness":
"low"}}

        # Build reasoning chains from timeline
        reasoning_chains = []
        for event in parsed.get("timeline", []):
            if event.get("significance") in ("concerning", "critical"):
                reasoning_chains.append(
                    f"{event.get('date', 'Unknown date')}: {event['event']} [{event['significance']}]"
                )

        # Extract recommendations
        recommendations = []
        trajectory = parsed.get("health_trajectory", {})
        for concern in trajectory.get("areas_of_concern", []):
            recommendations.append(f"Monitor: {concern}")

        avg_confidence = self._calc_avg_confidence(graph)
```

```
        return AgentResponse(
            agent_name=self.agent_name,
            response=parsed.get("narrative", ""),
            confidence=avg_confidence,
            reasoning_chains=reasoning_chains,
            alerts=[],
            care_gaps=[],
            recommendations=recommendations,
            graph_nodes_used=graph["summary"]["total_nodes"],
            metadata={
                "timeline": parsed.get("timeline", []),
                "providers": parsed.get("providers", []),
                "health_trajectory": trajectory,
                "data_quality": parsed.get("data_quality", {})
            }
        )

    def _format_graph(self, graph: dict) -> str:
        """Format the full patient graph for LLM consumption."""
        sections = []

        # Conditions
        conditions = graph.get("nodes_by_type", {}).get("condition", [])
        if conditions:
            lines = ["=== ACTIVE CONDITIONS ==="]
            for c in conditions:
                emoji = self._confidence_emoji(c.get("confidence_score", 0))
                lines.append(
                    f"- {c['display_name']} (code: {c.get('clinical_code', 'N/A')}) "
                    f"{emoji} confidence: {c.get('confidence_score', 0):.2f} "
                    f"source: {c.get('confidence_source', 'unknown')} "
                    f"since: {c.get('effective_start', 'unknown')}"
                )
            sections.append("\n".join(lines))

        # Medications
        medications = graph.get("nodes_by_type", {}).get("medication", [])
        if medications:
            lines = ["=== CURRENT MEDICATIONS ==="]
            for m in medications:
                emoji = self._confidence_emoji(m.get("confidence_score", 0))
                dosage = ""
                fhir = m.get("fhir_resource", {})
                if isinstance(fhir, dict):
                    dosage_list = fhir.get("dosage", [])
                    if dosage_list and isinstance(dosage_list, list):
                        dosage = dosage_list[0].get("text", "")
                prescriber = ""
```

```
            info_source = fhir.get("informationSource", {})
            if isinstance(info_source, dict):
                prescriber = info_source.get("display", "")

            lines.append(
                f"- {m['display_name']} | dosage: {dosage} | prescribed by: {prescriber or
'unknown'} "
                f"| {emoji} confidence: {m.get('confidence_score', 0):.2f} "
                f"| source: {m.get('confidence_source', 'unknown')} "
                f"| start: {m.get('effective_start', 'unknown')}"
            )
        sections.append("\n".join(lines))

    # Lab values
    labs = graph.get("nodes_by_type", {}).get("lab_value", [])
    if labs:
        lines = ["=== LAB VALUES ==="]
        for l in labs:
            emoji = self._confidence_emoji(l.get("confidence_score", 0))
            fhir = l.get("fhir_resource", {})
            interpretation = ""
            interp_list = fhir.get("interpretation", [])
            if interp_list and isinstance(interp_list, list):
                coding = interp_list[0].get("coding", [])
                if coding:
                    code = coding[0].get("code", "")
                    interpretation = " ⚠️ ABNORMAL" if code in ("H", "L") else " ✓ Normal"

            lines.append(
                f"- {l['display_name']}{interpretation} "
                f"| {emoji} confidence: {l.get('confidence_score', 0):.2f} "
                f"| date: {l.get('effective_start', 'unknown')}"
            )
        sections.append("\n".join(lines))

    # Care gaps
    care_gaps = graph.get("nodes_by_type", {}).get("care_gap", [])
    if care_gaps:
        lines = ["=== CARE GAPS (Overdue Screenings) ==="]
        for g in care_gaps:
            lines.append(f"- {g['display_name']}")
        sections.append("\n".join(lines))

    # Interactions (edges)
    edges = graph.get("edges", [])
    interactions = [e for e in edges if e.get("edge_type") == "interacts_with"]
    if interactions:
        lines = ["=== DRUG INTERACTIONS ==="]
```

```
            for i in interactions:
                props = i.get("properties", {})
                lines.append(
                    f"- {i.get('source_name', '?')} ↔ {i.get('target_name', '?')} "
                    f"| severity: {props.get('severity', 'unknown')} "
                    f"| {props.get('description', '')}"
                )
            sections.append("\n".join(lines))

        # Summary stats
        summary = graph.get("summary", {})
        sections.append(
            f"\n=== GRAPH STATS ===\n"
            f"Total nodes: {summary.get('total_nodes', 0)} | "
            f"Conditions: {summary.get('conditions', 0)} | "
            f"Medications: {summary.get('medications', 0)} | "
            f"Lab values: {summary.get('lab_values', 0)} | "
            f"Care gaps: {summary.get('care_gaps', 0)} | "
            f"Providers: {summary.get('providers', 0)} | "
            f"Interactions: {summary.get('interactions', 0)}"
        )

        return "\n\n".join(sections)

    def _confidence_emoji(self, score: float) -> str:
        if score >= 0.85: return "🟢"
        if score >= 0.65: return "🟡"
        if score >= 0.40: return "🟠"
        return "🔴"

    def _calc_avg_confidence(self, graph: dict) -> float:
        all_nodes = []
        for node_type, nodes in graph.get("nodes_by_type", {}).items():
            all_nodes.extend(nodes)
        if not all_nodes:
            return 0.0
        scores = [n.get("confidence_score", 0) for n in all_nodes]
        return sum(scores) / len(scores)


history_agent = HistoryAgent()
```

---

2. FAMILY CAREGIVER VIEW (Complete Implementation)

2.1 Updated Caregiver Routes (with Dashboard Data)

```
# api/routes/caregivers.py — Extended with family dashboard

from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel, EmailStr
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.feature_gate import get_user_tier
from api.middleware.audit import log_audit
from graph.phig_builder import phig_builder
from database.connection import async_session
from sqlalchemy import text
from typing import Optional
from utils.tier_config import check_feature_allowed

router = APIRouter()


class AddCaregiverRequest(BaseModel):
    caregiver_email: EmailStr
    relationship: str
    permission_level: str = "view"


@router.post("/add")
async def add_caregiver(req: AddCaregiverRequest, request: Request,
                        current_user=Depends(get_current_user)):
    """Patient adds a caregiver to their account."""
    patient_id = current_user["id"]
    tier = await get_user_tier(patient_id)

    # Check caregiver limit based on tier
    from utils.tier_config import get_limit
    max_caregivers = get_limit(tier, "max_caregivers")

    async with async_session() as session:
        count_result = await session.execute(text("""
            SELECT COUNT(*) FROM caregiver_links
            WHERE patient_user_id = :pid AND is_active = TRUE
        """), {"pid": patient_id})
        current_count = count_result.scalar()

    if max_caregivers and current_count >= max_caregivers:
        raise HTTPException(429, {
            "error": f"Caregiver limit reached ({current_count}/{max_caregivers})",
            "tier": tier,
            "upgrade_message": "Upgrade to add more caregivers."
```

```
        })

    if req.relationship not in ("son", "daughter", "spouse", "parent", "sibling", "other"):
        raise HTTPException(400, "Invalid relationship type")
    if req.permission_level not in ("view", "edit", "full"):
        raise HTTPException(400, "Invalid permission level")

    async with async_session() as session:
        result = await session.execute(
            text("SELECT id, name FROM users WHERE email = :email AND is_active =
TRUE"),
            {"email": req.caregiver_email}
        )
        caregiver = result.mappings().first()

    if not caregiver:
        raise HTTPException(404, "No registered user found with that email.")

    caregiver_id = str(caregiver["id"])
    if caregiver_id == patient_id:
        raise HTTPException(400, "You cannot add yourself as a caregiver")

    async with async_session() as session:
        await session.execute(text("""
            INSERT INTO caregiver_links (caregiver_user_id, patient_user_id,
                                         relationship, permission_level)
            VALUES (:cid, :pid, :rel, :perm)
            ON CONFLICT (caregiver_user_id, patient_user_id)
            DO UPDATE SET relationship = :rel, permission_level = :perm,
                          is_active = TRUE, revoked_at = NULL
        """), {"cid": caregiver_id, "pid": patient_id,
               "rel": req.relationship, "perm": req.permission_level})
        await session.commit()

    await log_audit(patient_id, patient_id, "ADD_CAREGIVER", request=request,
                    metadata={"caregiver_id": caregiver_id, "permission": req.permission_level})

    return {"status": "added", "caregiver_name": caregiver["name"],
            "permission_level": req.permission_level}


@router.delete("/{caregiver_id}")
async def revoke_caregiver(caregiver_id: str, request: Request,
                           current_user=Depends(get_current_user)):
    patient_id = current_user["id"]
    async with async_session() as session:
        await session.execute(text("""
            UPDATE caregiver_links SET is_active = FALSE, revoked_at = NOW()
```

```
            WHERE caregiver_user_id = :cid AND patient_user_id = :pid
        """), {"cid": caregiver_id, "pid": patient_id})
        await session.commit()

    await log_audit(patient_id, patient_id, "REVOKE_CAREGIVER", request=request)
    return {"status": "revoked"}


@router.get("/my-patients")
async def my_patients(current_user=Depends(get_current_user)):
    """Caregiver views list of patients they manage."""
    async with async_session() as session:
        result = await session.execute(text("""
            SELECT cl.patient_user_id, cl.relationship, cl.permission_level,
                   u.name AS patient_name
            FROM caregiver_links cl
            JOIN users u ON u.id = cl.patient_user_id
            WHERE cl.caregiver_user_id = :uid AND cl.is_active = TRUE
        """), {"uid": current_user["id"]})
        patients = result.mappings().all()

    return [dict(p) for p in patients]


@router.get("/my-caregivers")
async def my_caregivers(current_user=Depends(get_current_user)):
    """Patient views who has access to their data."""
    async with async_session() as session:
        result = await session.execute(text("""
            SELECT cl.caregiver_user_id, cl.relationship, cl.permission_level,
                   u.name AS caregiver_name, u.email AS caregiver_email,
                   cl.granted_at
            FROM caregiver_links cl
            JOIN users u ON u.id = cl.caregiver_user_id
            WHERE cl.patient_user_id = :uid AND cl.is_active = TRUE
            ORDER BY cl.granted_at DESC
        """), {"uid": current_user["id"]})
        caregivers = result.mappings().all()

    return [dict(c) for c in caregivers]


@router.get("/family-dashboard")
async def family_dashboard(request: Request, current_user=Depends(get_current_user)):
    """
    Unified family health dashboard.
    Returns health overview for ALL patients this caregiver manages.
    Premium Family feature.
```

```
    """
    tier = await get_user_tier(current_user["id"])
    if not check_feature_allowed(tier, "family_dashboard"):
        raise HTTPException(403, {
            "error": "Family dashboard requires Premium Family plan",
            "tier": tier,
            "upgrade_message": "Upgrade to Family Premium ($19.99/mo) for unified family
health view."
        })

    # Get all linked patients
    async with async_session() as session:
        result = await session.execute(text("""
            SELECT cl.patient_user_id, cl.relationship, cl.permission_level,
                   u.name AS patient_name,
                   pp.date_of_birth, pp.gender
            FROM caregiver_links cl
            JOIN users u ON u.id = cl.patient_user_id
            JOIN patient_profiles pp ON pp.user_id = u.id
            WHERE cl.caregiver_user_id = :uid AND cl.is_active = TRUE
        """), {"uid": current_user["id"]})
        patients = result.mappings().all()

    if not patients:
        return {"family_members": [], "urgent_alerts": [], "upcoming_actions": []}

    # Build family dashboard data
    family_members = []
    all_urgent_alerts = []
    all_upcoming_actions = []

    for patient in patients:
        pid = str(patient["patient_user_id"])

        # Get health overview
        graph = await phig_builder.get_full_patient_graph(pid)
        summary = graph.get("summary", {})

        # Get active alerts
        async with async_session() as session:
            alerts_result = await session.execute(text("""
                SELECT e.properties, s.display_name as drug_a, t.display_name as drug_b,
                       e.confidence_score
                FROM phig_edges e
                JOIN phig_nodes s ON s.id = e.source_node_id
                JOIN phig_nodes t ON t.id = e.target_node_id
                WHERE e.patient_id = :pid AND e.edge_type = 'interacts_with' AND e.is_active =
TRUE
```

```
            """), {"pid": pid})
            interaction_rows = alerts_result.mappings().all()

        patient_alerts = []
        for row in interaction_rows:
            import json as json_lib
            props = row["properties"] if isinstance(row["properties"], dict) else
json_lib.loads(row["properties"])
            alert = {
                "patient_name": patient["patient_name"],
                "patient_id": pid,
                "type": "drug_interaction",
                "severity": props.get("severity", "Unknown"),
                "drug_a": row["drug_a"],
                "drug_b": row["drug_b"],
                "description": props.get("description", ""),
                "clinical_action": props.get("clinical_action", ""),
            }
            patient_alerts.append(alert)
            if props.get("severity") in ("High", "CRITICAL", "ELEVATED"):
                all_urgent_alerts.append(alert)

        # Get care gaps as upcoming actions
        care_gaps = await phig_builder.evaluate_care_gaps(pid)
        for gap in care_gaps:
            all_upcoming_actions.append({
                "patient_name": patient["patient_name"],
                "patient_id": pid,
                "action": gap["screening_name"],
                "reason": gap.get("recommendation", ""),
                "urgency": "high" if gap.get("overdue_by_months", 0) and
gap["overdue_by_months"] > 6 else "medium",
                "overdue_months": gap.get("overdue_by_months")
            })

        # Get recent adherence (last 7 days)
        async with async_session() as session:
            adherence_result = await session.execute(text("""
                SELECT
                    COUNT(*) FILTER (WHERE response = 'taken') AS taken,
                    COUNT(*) FILTER (WHERE response = 'skipped') AS skipped,
                    COUNT(*) FILTER (WHERE response = 'no_response') AS missed,
                    COUNT(*) AS total
                FROM adherence_log
                WHERE patient_id = :pid
                  AND scheduled_time >= NOW() - INTERVAL '7 days'
            """), {"pid": pid})
            adherence = adherence_result.mappings().first()
```

```
    adherence_rate = 0.0
    if adherence and adherence["total"] > 0:
        adherence_rate = adherence["taken"] / adherence["total"]

    # Calculate age
    age = None
    if patient["date_of_birth"]:
        from datetime import date
        today = date.today()
        dob = patient["date_of_birth"]
        age = today.year - dob.year - ((today.month, today.day) < (dob.month, dob.day))

    family_members.append({
        "patient_id": pid,
        "name": patient["patient_name"],
        "relationship": patient["relationship"],
        "age": age,
        "gender": patient["gender"],
        "health_summary": {
            "conditions": summary.get("conditions", 0),
            "medications": summary.get("medications", 0),
            "active_alerts": len(patient_alerts),
            "care_gaps": summary.get("care_gaps", 0),
            "adherence_rate_7d": round(adherence_rate * 100, 1),
        },
        "alerts": patient_alerts,
        "status": _determine_patient_status(patient_alerts, care_gaps, adherence_rate)
    })

# Sort urgent alerts by severity
severity_order = {"CRITICAL": 0, "ELEVATED": 1, "High": 2, "Moderate": 3}
all_urgent_alerts.sort(key=lambda a: severity_order.get(a["severity"], 5))

# Sort upcoming actions by urgency
all_upcoming_actions.sort(key=lambda a: -(a.get("overdue_months") or 0))

await log_audit(current_user["id"], current_user["id"],
                "VIEW_FAMILY_DASHBOARD", request=request)

return {
    "family_members": family_members,
    "urgent_alerts": all_urgent_alerts,
    "upcoming_actions": all_upcoming_actions[:10],  # Top 10 most urgent
    "family_size": len(family_members)
}
```

```
def _determine_patient_status(alerts: list, care_gaps: list, adherence_rate: float) -> str:
    """Determine overall patient status for dashboard indicator."""
    critical_alerts = [a for a in alerts if a.get("severity") in ("CRITICAL", "ELEVATED", "High")]
    if critical_alerts:
        return "needs_attention"       # Red indicator
    if len(care_gaps) > 2 or adherence_rate < 0.5:
        return "monitor"               # Yellow indicator
    return "stable"                    # Green indicator
```

2.2 Family Dashboard Frontend

```
// src/app/(dashboard)/family/page.tsx

'use client';
import { useEffect, useState } from 'react';
import { Heart, AlertTriangle, Calendar, Pill, ChevronRight, Shield } from 'lucide-react';
import api from '@/lib/api';
import ConfidenceBadge from '@/components/ui/ConfidenceBadge';

interface FamilyMember {
  patient_id: string;
  name: string;
  relationship: string;
  age: number | null;
  gender: string;
  health_summary: {
    conditions: number;
    medications: number;
    active_alerts: number;
    care_gaps: number;
    adherence_rate_7d: number;
  };
  alerts: any[];
  status: 'stable' | 'monitor' | 'needs_attention';
}

const STATUS_CONFIG = {
  stable:          { color: 'bg-green-100 text-green-800', dot: 'bg-green-500', label: 'Stable' },
  monitor:         { color: 'bg-yellow-100 text-yellow-800', dot: 'bg-yellow-500', label: 'Monitor' },
  needs_attention: { color: 'bg-red-100 text-red-800', dot: 'bg-red-500', label: 'Needs Attention'
},
};

export default function FamilyDashboardPage() {
  const [data, setData] = useState<any>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
```

```
  useEffect(() => {
    api.get('/caregivers/family-dashboard')
      .then(res => { setData(res.data); setLoading(false); })
      .catch(err => {
        if (err.response?.status === 403) {
          setError('Family dashboard requires Premium Family plan.');
        } else {
          setError('Failed to load family dashboard.');
        }
        setLoading(false);
      });
  }, []);

  if (loading) return <div className="p-8 text-center text-gray-500">Loading family health
data...</div>;
  if (error) return (
    <div className="p-8 text-center">
      <Shield className="w-12 h-12 text-gray-300 mx-auto mb-3" />
      <p className="text-gray-600">{error}</p>
      <a href="/settings/subscription" className="text-blue-600 hover:underline mt-2
inline-block">
        Upgrade to Family Premium →
      </a>
    </div>
  );

  const { family_members, urgent_alerts, upcoming_actions } = data;

  return (
    <div className="p-6 space-y-6 max-w-5xl mx-auto">
      <h1 className="text-2xl font-bold text-gray-800">Family Health Dashboard</h1>
      <p className="text-gray-500">Managing {family_members.length} family
member(s)</p>

      {/* Urgent alerts banner */}
      {urgent_alerts.length > 0 && (
        <div className="bg-red-50 border border-red-200 rounded-xl p-4">
          <h2 className="text-lg font-semibold text-red-800 flex items-center gap-2 mb-3">
            <AlertTriangle className="w-5 h-5" /> Urgent Alerts
          </h2>
          {urgent_alerts.map((alert: any, i: number) => (
            <div key={i} className="bg-white rounded-lg p-3 border border-red-100 mb-2">
              <div className="flex items-center gap-2 mb-1">
                <span className="text-xs bg-red-100 text-red-700 px-2 py-0.5 rounded-full
font-medium">
                  {alert.patient_name}
                </span>
```

```
            <span className="text-xs text-red-600 font-medium">{alert.severity}</span>
          </div>
          <p className="text-sm font-medium text-gray-800">
            {alert.drug_a} + {alert.drug_b}
          </p>
          <p className="text-sm text-gray-600 mt-1">{alert.clinical_action}</p>
        </div>
      ))}
    </div>
  )}

  {/* Family member cards */}
  <div className="grid md:grid-cols-2 gap-4">
    {family_members.map((member: FamilyMember) => {
      const statusConfig = STATUS_CONFIG[member.status];
      return (
        <div key={member.patient_id}
             className="bg-white rounded-xl border shadow-sm p-5 hover:shadow-md
transition-shadow cursor-pointer"
             onClick={() => window.location.href =
`/dashboard?patient_id=${member.patient_id}`}>

          {/* Header */}
          <div className="flex items-center justify-between mb-4">
            <div>
              <h3 className="font-semibold text-gray-800">{member.name}</h3>
              <p className="text-sm text-gray-500">
                {member.relationship}{member.age ? `, ${member.age} years` : ''}
              </p>
            </div>
            <span className={`px-3 py-1 rounded-full text-xs font-medium flex items-center
gap-1.5 ${statusConfig.color}`}>
              <span className={`w-2 h-2 rounded-full ${statusConfig.dot}`}></span>
              {statusConfig.label}
            </span>
          </div>

          {/* Stats grid */}
          <div className="grid grid-cols-4 gap-3 mb-4">
            <div className="text-center">
              <p className="text-xl font-bold
text-gray-800">{member.health_summary.conditions}</p>
              <p className="text-xs text-gray-500">Conditions</p>
            </div>
            <div className="text-center">
              <p className="text-xl font-bold
text-gray-800">{member.health_summary.medications}</p>
              <p className="text-xs text-gray-500">Medications</p>
```

```
            </div>
            <div className="text-center">
              <p className={`text-xl font-bold ${member.health_summary.active_alerts > 0 ?
'text-red-600' : 'text-gray-800'}`}>
                {member.health_summary.active_alerts}
              </p>
              <p className="text-xs text-gray-500">Alerts</p>
            </div>
            <div className="text-center">
              <p className={`text-xl font-bold ${
                member.health_summary.adherence_rate_7d >= 80 ? 'text-green-600' :
                member.health_summary.adherence_rate_7d >= 50 ? 'text-yellow-600' :
'text-red-600'
              }`}>
                {member.health_summary.adherence_rate_7d}%
              </p>
              <p className="text-xs text-gray-500">Adherence</p>
            </div>
          </div>

          {/* Alerts preview */}
          {member.alerts.length > 0 && (
            <div className="bg-red-50 rounded-lg p-2 mb-3">
              <p className="text-xs text-red-700 font-medium">
                ⚠️ {member.alerts[0].drug_a} + {member.alerts[0].drug_b} interaction
              </p>
            </div>
          )}

          {/* Care gaps preview */}
          {member.health_summary.care_gaps > 0 && (
            <div className="bg-orange-50 rounded-lg p-2 mb-3">
              <p className="text-xs text-orange-700 font-medium">
                📋 {member.health_summary.care_gaps} overdue screening(s)
              </p>
            </div>
          )}

          <div className="flex items-center justify-end text-sm text-blue-600">
            View details <ChevronRight className="w-4 h-4" />
          </div>
        </div>
      );
    })}
  </div>

  {/* Upcoming actions */}
  {upcoming_actions.length > 0 && (
```

```
        <div className="bg-blue-50 border border-blue-200 rounded-xl p-4">
          <h2 className="text-lg font-semibold text-blue-800 flex items-center gap-2 mb-3">
            <Calendar className="w-5 h-5" /> Upcoming Actions
          </h2>
          <div className="space-y-2">
            {upcoming_actions.slice(0, 5).map((action: any, i: number) => (
              <div key={i} className="bg-white rounded-lg p-3 border border-blue-100 flex
items-center justify-between">
                <div>
                  <span className="text-xs bg-blue-100 text-blue-700 px-2 py-0.5 rounded-full
font-medium mr-2">
                    {action.patient_name}
                  </span>
                  <span className="text-sm font-medium text-gray-800">{action.action}</span>
                  {action.overdue_months && (
                    <span className="text-xs text-red-600 ml-2">
                      {action.overdue_months} months overdue
                    </span>
                  )}
                </div>
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
  );
}
```

---

3. LAB VALUE TREND VISUALIZATION

3.1 Lab Trends API Route

```
# api/routes/labs.py

from fastapi import APIRouter, Depends, HTTPException, Request, Query
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.audit import log_audit
from database.connection import async_session
from sqlalchemy import text
from typing import Optional

router = APIRouter()
```

```
@router.get("/trends")
async def get_lab_trends(
    request: Request,
    patient_id: str = None,
    test_name: str = Query(None, description="Filter by test name, e.g. 'HbA1c'"),
    loinc_code: str = Query(None, description="Filter by LOINC code, e.g. '4548-4'"),
    months: int = Query(24, ge=1, le=60, description="How many months of history"),
    current_user=Depends(get_current_user)
):
    """
    Get lab value trends for visualization.
    Returns time-series data grouped by test type.
    """
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    # Build filter
    filters = ["n.patient_id = :pid", "n.node_type = 'lab_value'", "n.is_active = TRUE"]
    params = {"pid": target, "months": months}

    if loinc_code:
        filters.append("n.clinical_code = :code")
        params["code"] = loinc_code
    elif test_name:
        filters.append("n.display_name ILIKE :name")
        params["name"] = f"%{test_name}%"

    where_clause = " AND ".join(filters)

    async with async_session() as session:
        result = await session.execute(text(f"""
            SELECT
                n.id,
                n.display_name,
                n.clinical_code AS loinc_code,
                n.confidence_score,
                n.confidence_source,
                n.effective_start AS test_date,
                n.fhir_resource
            FROM phig_nodes n
            WHERE {where_clause}
              AND n.effective_start >= (CURRENT_DATE - (:months || ' months')::INTERVAL)
            ORDER BY n.clinical_code, n.effective_start ASC
        """), params)
        rows = result.mappings().all()

    # Group by test type and extract values
```

```
trends = {}
for row in rows:
    fhir = row["fhir_resource"] if isinstance(row["fhir_resource"], dict) else {}
    value_quantity = fhir.get("valueQuantity", {})
    value = value_quantity.get("value")
    unit = value_quantity.get("unit", "")

    # Extract reference range
    ref_ranges = fhir.get("referenceRange", [])
    ref_low = None
    ref_high = None
    if ref_ranges:
        ref_low = ref_ranges[0].get("low", {}).get("value")
        ref_high = ref_ranges[0].get("high", {}).get("value")

    # Determine if abnormal
    interpretation = fhir.get("interpretation", [])
    is_abnormal = False
    if interpretation:
        coding = interpretation[0].get("coding", [])
        if coding:
            is_abnormal = coding[0].get("code") in ("H", "L", "HH", "LL")

    # Parse display name for grouping key
    # "HbA1c: 7.8 %" → "HbA1c"
    test_key = row["display_name"].split(":")[0].strip()
    loinc = row["loinc_code"] or test_key

    if loinc not in trends:
        trends[loinc] = {
            "test_name": test_key,
            "loinc_code": row["loinc_code"],
            "unit": unit,
            "reference_range": {"low": ref_low, "high": ref_high},
            "data_points": []
        }

    if value is not None:
        trends[loinc]["data_points"].append({
            "date": str(row["test_date"]) if row["test_date"] else None,
            "value": float(value),
            "is_abnormal": is_abnormal,
            "confidence": float(row["confidence_score"]),
            "source": row["confidence_source"]
        })

# Calculate trend direction for each test
for loinc, trend_data in trends.items():
```

```
        points = trend_data["data_points"]
        if len(points) >= 2:
            first_val = points[0]["value"]
            last_val = points[-1]["value"]
            ref_high = trend_data["reference_range"]["high"]
            ref_low = trend_data["reference_range"]["low"]

            change_pct = ((last_val - first_val) / first_val * 100) if first_val != 0 else 0

            # Determine if change is clinically significant
            if abs(change_pct) < 5:
                direction = "stable"
            elif change_pct > 0:
                # Rising — bad if above ref range, potentially good if below
                if ref_high and last_val > ref_high:
                    direction = "worsening"
                elif ref_low and last_val < ref_low:
                    direction = "improving"  # Rising toward normal
                else:
                    direction = "rising"
            else:
                # Falling — bad if below ref range (e.g., eGFR), good if above
                if ref_low and last_val < ref_low:
                    direction = "worsening"
                elif ref_high and last_val > ref_high:
                    direction = "improving"
                else:
                    direction = "falling"

            trend_data["trend"] = {
                "direction": direction,
                "change_percent": round(change_pct, 1),
                "data_point_count": len(points)
            }
        else:
            trend_data["trend"] = {
                "direction": "insufficient_data",
                "change_percent": 0,
                "data_point_count": len(points)
            }

    await log_audit(current_user["id"], target, "VIEW_LAB_TRENDS", request=request)

    return {"trends": list(trends.values())}


@router.get("/available-tests")
async def get_available_tests(
```

```
    patient_id: str = None,
    current_user=Depends(get_current_user)
):
    """List all lab test types available for this patient."""
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    async with async_session() as session:
        result = await session.execute(text("""
            SELECT DISTINCT
                clinical_code AS loinc_code,
                SPLIT_PART(display_name, ':', 1) AS test_name,
                COUNT(*) AS data_points,
                MIN(effective_start) AS first_test,
                MAX(effective_start) AS last_test
            FROM phig_nodes
            WHERE patient_id = :pid AND node_type = 'lab_value' AND is_active = TRUE
            GROUP BY clinical_code, SPLIT_PART(display_name, ':', 1)
            ORDER BY MAX(effective_start) DESC
        """), {"pid": target})
        tests = result.mappings().all()

    return [dict(t) for t in tests]
```

3.2 Lab Trends Visualization Component

```
// src/app/(dashboard)/labs/page.tsx

'use client';
import { useEffect, useState } from 'react';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip,
         ReferenceLine, ResponsiveContainer, Area, ComposedChart } from 'recharts';
import { TrendingUp, TrendingDown, Minus, AlertTriangle } from 'lucide-react';
import api from '@/lib/api';

interface TrendData {
  test_name: string;
  loinc_code: string;
  unit: string;
  reference_range: { low: number | null; high: number | null };
  data_points: Array<{
    date: string;
    value: number;
    is_abnormal: boolean;
    confidence: number;
  }>;
  trend: {
```

```
    direction: string;
    change_percent: number;
    data_point_count: number;
  };
}

const TREND_ICONS: Record<string, { icon: any; color: string; label: string }> = {
  stable:     { icon: Minus, color: 'text-green-600', label: 'Stable' },
  improving:  { icon: TrendingUp, color: 'text-green-600', label: 'Improving' },
  worsening:  { icon: TrendingDown, color: 'text-red-600', label: 'Worsening' },
  rising:     { icon: TrendingUp, color: 'text-yellow-600', label: 'Rising' },
  falling:    { icon: TrendingDown, color: 'text-yellow-600', label: 'Falling' },
  insufficient_data: { icon: Minus, color: 'text-gray-400', label: 'Not enough data' },
};

export default function LabTrendsPage() {
  const [trends, setTrends] = useState<TrendData[]>([]);
  const [selectedTest, setSelectedTest] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.get('/labs/trends?months=24').then(res => {
      setTrends(res.data.trends);
      if (res.data.trends.length > 0) {
        setSelectedTest(res.data.trends[0].loinc_code || res.data.trends[0].test_name);
      }
      setLoading(false);
    });
  }, []);

  if (loading) return <div className="p-8 text-center text-gray-500">Loading lab
trends...</div>;
  if (trends.length === 0) return (
    <div className="p-8 text-center text-gray-500">
      No lab results yet. Upload a lab report to start tracking trends.
    </div>
  );

  const activeTrend = trends.find(t =>
    (t.loinc_code || t.test_name) === selectedTest
  );

  return (
    <div className="p-6 space-y-6 max-w-5xl mx-auto">
      <h1 className="text-2xl font-bold text-gray-800">Lab Value Trends</h1>

      {/* Test selector pills */}
      <div className="flex flex-wrap gap-2">
```

```
        {trends.map(trend => {
          const key = trend.loinc_code || trend.test_name;
          const isActive = key === selectedTest;
          const trendInfo = TREND_ICONS[trend.trend?.direction] ||
TREND_ICONS.insufficient_data;

          return (
            <button
              key={key}
              onClick={() => setSelectedTest(key)}
              className={`px-4 py-2 rounded-lg text-sm font-medium transition-all
                      flex items-center gap-2
                      ${isActive
                        ? 'bg-blue-600 text-white shadow-md'
                        : 'bg-gray-100 text-gray-700 hover:bg-gray-200'}`}
            >
              {trend.test_name}
              {trend.trend?.direction === 'worsening' && (
                <AlertTriangle className="w-3.5 h-3.5 text-red-300" />
              )}
            </button>
          );
        })}
      </div>

      {/* Chart */}
      {activeTrend && (
        <div className="bg-white rounded-xl border shadow-sm p-6">
          {/* Header */}
          <div className="flex items-center justify-between mb-4">
            <div>
              <h2 className="text-lg font-semibold
text-gray-800">{activeTrend.test_name}</h2>
              <p className="text-sm text-gray-500">
                {activeTrend.data_points.length} results
                {activeTrend.unit && ` · Unit: ${activeTrend.unit}`}
                {activeTrend.reference_range.low != null && activeTrend.reference_range.high !=
null && (
                  <> · Normal:
{activeTrend.reference_range.low}–{activeTrend.reference_range.high}</>
                )}
              </p>
            </div>
            {activeTrend.trend && (() => {
              const trendInfo = TREND_ICONS[activeTrend.trend.direction] ||
TREND_ICONS.insufficient_data;
              const TrendIcon = trendInfo.icon;
              return (
```

```
            <div className={`flex items-center gap-2 px-3 py-1.5 rounded-lg ${
              activeTrend.trend.direction === 'worsening' ? 'bg-red-50' :
              activeTrend.trend.direction === 'improving' ? 'bg-green-50' : 'bg-gray-50'
            }`}>
              <TrendIcon className={`w-4 h-4 ${trendInfo.color}`} />
              <span className={`text-sm font-medium ${trendInfo.color}`}>
                {trendInfo.label}
                {activeTrend.trend.change_percent !== 0 && (
                  <> ({activeTrend.trend.change_percent > 0 ? '+' :
''}{activeTrend.trend.change_percent}%)</>
                )}
              </span>
            </div>
          );
        })()}
      </div>

      {/* Recharts visualization */}
      <ResponsiveContainer width="100%" height={300}>
        <ComposedChart data={activeTrend.data_points.map(p => ({
          ...p,
          date: new Date(p.date).toLocaleDateString('en-IN', { month: 'short', year: '2-digit' })
        }))}>
          <CartesianGrid strokeDasharray="3 3" stroke="#f0f0f0" />
          <XAxis dataKey="date" tick={{ fontSize: 12 }} />
          <YAxis tick={{ fontSize: 12 }} domain={['auto', 'auto']} />
          <Tooltip
            content={({ active, payload }) => {
              if (!active || !payload?.length) return null;
              const d = payload[0].payload;
              return (
                <div className="bg-white shadow-lg rounded-lg p-3 border text-sm">
                  <p className="font-medium">{d.date}</p>
                  <p className={d.is_abnormal ? 'text-red-600 font-bold' : 'text-gray-800'}>
                    {d.value} {activeTrend.unit} {d.is_abnormal ? ' ⚠️ Abnormal' : ''}
                  </p>
                  <p className="text-gray-500 text-xs">Confidence: {(d.confidence *
100).toFixed(0)}%</p>
                </div>
              );
            }}
          />

          {/* Reference range shading */}
          {activeTrend.reference_range.low != null && (
            <ReferenceLine y={activeTrend.reference_range.low}
                           stroke="#22c55e" strokeDasharray="5 5"
                           label={{ value: 'Low', position: 'left', fontSize: 10 }} />
```

```
            )}
            {activeTrend.reference_range.high != null && (
              <ReferenceLine y={activeTrend.reference_range.high}
                            stroke="#ef4444" strokeDasharray="5 5"
                            label={{ value: 'High', position: 'left', fontSize: 10 }} />
            )}

            <Line type="monotone" dataKey="value" stroke="#2563eb"
                 strokeWidth={2} dot={{ r: 5, fill: '#2563eb' }}
                 activeDot={{ r: 7 }} />
          </ComposedChart>
        </ResponsiveContainer>
      </div>
    )}
  </div>
 );
}
```

---

4. MEDICATION ADHERENCE TRACKING

4.1 Adherence API Routes

```
# api/routes/adherence.py

from fastapi import APIRouter, Depends, HTTPException, Request, Query
from pydantic import BaseModel
from api.middleware.auth import get_current_user
from api.middleware.rbac import verify_patient_access
from api.middleware.audit import log_audit
from database.connection import async_session
from sqlalchemy import text
from typing import Optional
from uuid import uuid4

router = APIRouter()


class RecordAdherenceRequest(BaseModel):
    reminder_id: str
    response: str          # taken, skipped, snoozed


@router.post("/record")
async def record_adherence(req: RecordAdherenceRequest, request: Request,
                           current_user=Depends(get_current_user)):
```

```
    """Record medication adherence response (from email link or web UI)."""
    if req.response not in ("taken", "skipped", "snoozed"):
        raise HTTPException(400, "Invalid response. Must be 'taken', 'skipped', or 'snoozed'.")

    # Get reminder details
    async with async_session() as session:
        result = await session.execute(text("""
            SELECT r.patient_id, r.medication_node_id, n.display_name
            FROM medication_reminders r
            JOIN phig_nodes n ON n.id = r.medication_node_id
            WHERE r.id = :rid AND r.is_active = TRUE
        """), {"rid": req.reminder_id})
        reminder = result.mappings().first()

    if not reminder:
        raise HTTPException(404, "Reminder not found")

    patient_id = str(reminder["patient_id"])
    await verify_patient_access(current_user["id"], patient_id, "edit")

    # Record adherence log
    async with async_session() as session:
        await session.execute(text("""
            INSERT INTO adherence_log (id, reminder_id, patient_id, medication_node_id,
                                       scheduled_time, response, responded_at, response_channel)
            VALUES (:id, :rid, :pid, :mid, NOW(), :response, NOW(), 'web')
        """), {
            "id": str(uuid4()),
            "rid": req.reminder_id,
            "pid": patient_id,
            "mid": str(reminder["medication_node_id"]),
            "response": req.response
        })

        # Update reminder last_confirmed_at if taken
        if req.response == "taken":
            await session.execute(text("""
                UPDATE medication_reminders SET last_confirmed_at = NOW()
                WHERE id = :rid
            """), {"rid": req.reminder_id})

        await session.commit()

    return {"status": "recorded", "response": req.response,
            "medication": reminder["display_name"]}


@router.get("/stats")
```

```
async def get_adherence_stats(
    request: Request,
    patient_id: str = None,
    days: int = Query(30, ge=1, le=365),
    current_user=Depends(get_current_user)
):
    """
    Get medication adherence statistics.
    Returns overall rate + per-medication breakdown.
    """
    target = patient_id or current_user["id"]
    await verify_patient_access(current_user["id"], target, "view")

    async with async_session() as session:
        # Overall stats
        overall = await session.execute(text("""
            SELECT
                COUNT(*) AS total,
                COUNT(*) FILTER (WHERE response = 'taken') AS taken,
                COUNT(*) FILTER (WHERE response = 'skipped') AS skipped,
                COUNT(*) FILTER (WHERE response = 'snoozed') AS snoozed,
                COUNT(*) FILTER (WHERE response = 'no_response' OR response IS NULL) AS
missed
            FROM adherence_log
            WHERE patient_id = :pid
              AND scheduled_time >= NOW() - (:days || ' days')::INTERVAL
        """), {"pid": target, "days": days})
        overall_stats = overall.mappings().first()

        # Per-medication breakdown
        per_med = await session.execute(text("""
            SELECT
                n.display_name AS medication_name,
                al.medication_node_id,
                COUNT(*) AS total,
                COUNT(*) FILTER (WHERE al.response = 'taken') AS taken,
                COUNT(*) FILTER (WHERE al.response = 'skipped') AS skipped,
                COUNT(*) FILTER (WHERE al.response = 'no_response' OR al.response IS
NULL) AS missed
            FROM adherence_log al
            JOIN phig_nodes n ON n.id = al.medication_node_id
            WHERE al.patient_id = :pid
              AND al.scheduled_time >= NOW() - (:days || ' days')::INTERVAL
            GROUP BY n.display_name, al.medication_node_id
            ORDER BY n.display_name
        """), {"pid": target, "days": days})
        per_med_stats = per_med.mappings().all()
```

```
        # Daily adherence for chart (last N days)
        daily = await session.execute(text("""
            SELECT
                DATE(scheduled_time) AS day,
                COUNT(*) AS total,
                COUNT(*) FILTER (WHERE response = 'taken') AS taken
            FROM adherence_log
            WHERE patient_id = :pid
              AND scheduled_time >= NOW() - (:days || ' days')::INTERVAL
            GROUP BY DATE(scheduled_time)
            ORDER BY DATE(scheduled_time)
        """), {"pid": target, "days": days})
        daily_stats = daily.mappings().all()

        # Current streak
        streak = await session.execute(text("""
            WITH daily_adherence AS (
                SELECT DATE(scheduled_time) AS day,
                    CASE WHEN COUNT(*) FILTER (WHERE response = 'taken') =
                        COUNT(*) THEN TRUE ELSE FALSE END AS all_taken
                FROM adherence_log
                WHERE patient_id = :pid
                GROUP BY DATE(scheduled_time)
                ORDER BY DATE(scheduled_time) DESC
            ),
            streak AS (
                SELECT day, all_taken,
                    ROW_NUMBER() OVER (ORDER BY day DESC) -
                    ROW_NUMBER() OVER (PARTITION BY all_taken ORDER BY day DESC)
AS grp
                FROM daily_adherence
            )
            SELECT COUNT(*) AS streak_days
            FROM streak
            WHERE all_taken = TRUE AND grp = (
                SELECT grp FROM streak WHERE all_taken = TRUE ORDER BY day DESC
LIMIT 1
            )
        """), {"pid": target})
        streak_result = streak.mappings().first()

    total = overall_stats["total"] if overall_stats["total"] > 0 else 1
    adherence_rate = overall_stats["taken"] / total

    await log_audit(current_user["id"], target, "VIEW_ADHERENCE", request=request)

    return {
        "period_days": days,
```

```
        "overall": {
            "total_doses": overall_stats["total"],
            "taken": overall_stats["taken"],
            "skipped": overall_stats["skipped"],
            "snoozed": overall_stats["snoozed"],
            "missed": overall_stats["missed"],
            "adherence_rate": round(adherence_rate * 100, 1),
            "current_streak_days": streak_result["streak_days"] if streak_result else 0,
        },
        "per_medication": [
            {
                "medication_name": m["medication_name"],
                "medication_node_id": str(m["medication_node_id"]),
                "total": m["total"],
                "taken": m["taken"],
                "adherence_rate": round(m["taken"] / max(m["total"], 1) * 100, 1)
            }
            for m in per_med_stats
        ],
        "daily": [
            {
                "date": str(d["day"]),
                "total": d["total"],
                "taken": d["taken"],
                "rate": round(d["taken"] / max(d["total"], 1) * 100, 1)
            }
            for d in daily_stats
        ]
    }
```

4.2 Adherence Dashboard Component

```
// src/app/(dashboard)/adherence/page.tsx

'use client';
import { useEffect, useState } from 'react';
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip,
         ResponsiveContainer, Cell } from 'recharts';
import { Flame, Pill, TrendingUp } from 'lucide-react';
import api from '@/lib/api';

export default function AdherencePage() {
  const [stats, setStats] = useState<any>(null);
  const [days, setDays] = useState(30);

  useEffect(() => {
    api.get(`/adherence/stats?days=${days}`).then(res => setStats(res.data));
```

```
  }, [days]);

  if (!stats) return <div className="p-8 text-center text-gray-500">Loading adherence
data...</div>;

  const overallRate = stats.overall.adherence_rate;
  const streak = stats.overall.current_streak_days;

  // Color based on adherence rate
  const rateColor = overallRate >= 80 ? 'text-green-600' :
                    overallRate >= 50 ? 'text-yellow-600' : 'text-red-600';
  const rateBg = overallRate >= 80 ? 'bg-green-50' :
                 overallRate >= 50 ? 'bg-yellow-50' : 'bg-red-50';

  return (
    <div className="p-6 space-y-6 max-w-5xl mx-auto">
      <h1 className="text-2xl font-bold text-gray-800">Medication Adherence</h1>

      {/* Period selector */}
      <div className="flex gap-2">
        {[7, 14, 30, 90].map(d => (
          <button key={d} onClick={() => setDays(d)}
                  className={`px-4 py-2 rounded-lg text-sm font-medium transition-colors
                              ${days === d ? 'bg-blue-600 text-white' : 'bg-gray-100 text-gray-600
hover:bg-gray-200'}`}>
            {d}d
          </button>
        ))}
      </div>

      {/* Summary cards */}
      <div className="grid grid-cols-3 gap-4">
        <div className={`rounded-xl p-5 ${rateBg}`}>
          <TrendingUp className={`w-6 h-6 ${rateColor} mb-2`} />
          <p className={`text-3xl font-bold ${rateColor}`}>{overallRate}%</p>
          <p className="text-sm text-gray-600 mt-1">Adherence Rate</p>
        </div>
        <div className="bg-orange-50 rounded-xl p-5">
          <Flame className="w-6 h-6 text-orange-500 mb-2" />
          <p className="text-3xl font-bold text-orange-600">{streak}</p>
          <p className="text-sm text-gray-600 mt-1">Day Streak 🔥</p>
        </div>
        <div className="bg-blue-50 rounded-xl p-5">
          <Pill className="w-6 h-6 text-blue-500 mb-2" />
          <p className="text-3xl font-bold text-blue-600">
            {stats.overall.taken}/{stats.overall.total_doses}
          </p>
          <p className="text-sm text-gray-600 mt-1">Doses Taken</p>
```

```
      </div>
    </div>

    {/* Daily adherence chart */}
    <div className="bg-white rounded-xl border shadow-sm p-6">
      <h2 className="text-lg font-semibold text-gray-800 mb-4">Daily Adherence</h2>
      <ResponsiveContainer width="100%" height={250}>
        <BarChart data={stats.daily}>
          <CartesianGrid strokeDasharray="3 3" stroke="#f0f0f0" />
          <XAxis dataKey="date" tick={{ fontSize: 11 }}
              tickFormatter={(d) => new Date(d).toLocaleDateString('en-IN', { day: 'numeric',
month: 'short' })} />
          <YAxis domain={[0, 100]} tick={{ fontSize: 11 }}
              tickFormatter={(v) => `${v}%`} />
          <Tooltip
            formatter={(value: number) => [`${value}%`, 'Adherence']}
            labelFormatter={(d) => new Date(d).toLocaleDateString('en-IN', { weekday: 'short',
day: 'numeric', month: 'short' })}
          />
          <Bar dataKey="rate" radius={[4, 4, 0, 0]}>
            {stats.daily.map((entry: any, i: number) => (
              <Cell key={i} fill={entry.rate >= 80 ? '#22c55e' : entry.rate >= 50 ? '#eab308' :
'#ef4444'} />
            ))}
          </Bar>
        </BarChart>
      </ResponsiveContainer>
    </div>

    {/* Per-medication breakdown */}
    <div className="bg-white rounded-xl border shadow-sm p-6">
      <h2 className="text-lg font-semibold text-gray-800 mb-4">By Medication</h2>
      <div className="space-y-4">
        {stats.per_medication.map((med: any) => {
          const rate = med.adherence_rate;
          const barColor = rate >= 80 ? 'bg-green-500' : rate >= 50 ? 'bg-yellow-500' :
'bg-red-500';
          return (
            <div key={med.medication_node_id}>
              <div className="flex items-center justify-between mb-1">
                <span className="text-sm font-medium
text-gray-700">{med.medication_name}</span>
                <span className={`text-sm font-bold ${
                  rate >= 80 ? 'text-green-600' : rate >= 50 ? 'text-yellow-600' : 'text-red-600'
                }`}>
                  {rate}%
                </span>
              </div>
```

```
            <div className="w-full bg-gray-100 rounded-full h-3">
              <div className={`h-3 rounded-full transition-all ${barColor}`}
                   style={{ width: `${rate}%` }} />
            </div>
            <p className="text-xs text-gray-500 mt-1">
              {med.taken} taken / {med.total} scheduled
            </p>
          </div>
        );
      })}
    </div>
  </div>

  {/* Streak celebration */}
  {streak >= 7 && (
    <div className="bg-gradient-to-r from-orange-100 to-yellow-100 rounded-xl p-6
text-center">
      <p className="text-4xl mb-2">🎉</p>
      <p className="text-lg font-bold text-orange-800">
        {streak}-Day Streak!
      </p>
      <p className="text-sm text-orange-600 mt-1">
        {streak >= 30 ? "Incredible consistency! Your health thanks you." :
         streak >= 14 ? "Two weeks strong! Keep it up." :
         "Great start! One week of perfect adherence."}
      </p>
    </div>
  )}
 </div>
 );
}
```

---

5. UPDATED main.py (All New Routes Registered)

```
# main.py — Updated with all P0 routes

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
from config import get_settings
from database.connection import init_db, close_db
from api.routes import (
    auth, patients, documents, chat, summary,
    caregivers, reminders, subscriptions,
    confirmations, labs, adherence
)
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    yield
    await close_db()

settings = get_settings()
app = FastAPI(title="CareOrbit API", version="0.3.0", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS.split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Auth
app.include_router(auth.router, prefix="/api/auth", tags=["Authentication"])

# Patient data
app.include_router(patients.router, prefix="/api/patients", tags=["Patients"])
app.include_router(labs.router, prefix="/api/labs", tags=["Lab Values"])
app.include_router(adherence.router, prefix="/api/adherence", tags=["Adherence"])

# Document processing
app.include_router(documents.router, prefix="/api/documents", tags=["Documents"])
app.include_router(confirmations.router, prefix="/api/confirmations", tags=["Confirmations"])

# AI features
app.include_router(chat.router, prefix="/api/chat", tags=["AI Chat"])
app.include_router(summary.router, prefix="/api/summary", tags=["Health Summary"])

# Social features
app.include_router(caregivers.router, prefix="/api/caregivers", tags=["Caregivers"])

# Notifications
app.include_router(reminders.router, prefix="/api/reminders", tags=["Reminders"])

# Billing
app.include_router(subscriptions.router, prefix="/api/subscriptions", tags=["Subscriptions"])

@app.get("/health")
async def health_check():
    return {"status": "healthy", "service": "careorbit-api", "version": "0.3.0"}
```

---

6. UPDATED PROJECT FILE MAP

```
careorbit-backend/
├── main.py                    # v0.3.0 — all routes registered
├── config.py
├── requirements.txt
├── Dockerfile
│
├── api/routes/
│   ├── auth.py
│   ├── patients.py
│   ├── documents.py           # With tier enforcement
│   ├── chat.py
│   ├── summary.py
│   ├── caregivers.py          # Extended with family-dashboard
│   ├── reminders.py
│   ├── confirmations.py       # Web-based data confirmation
│   ├── subscriptions.py       # Plans, upgrade, usage
│   ├── labs.py                # NEW: Lab trends + available tests
│   └── adherence.py           # NEW: Record + stats
│
├── api/middleware/
│   ├── auth.py
│   ├── rbac.py
│   ├── audit.py
│   └── feature_gate.py
│
├── agents/
│   ├── base_agent.py
│   ├── medication_agent.py
│   ├── care_gap_agent.py
│   ├── history_agent.py       # NOW P0 — complete implementation
│   └── orchestrator.py
│
├── (pipeline/, graph/, services/, utils/ — unchanged)
│
└── tests/
    ├── test_confidence.py
    ├── test_drug_database.py
    ├── test_auth.py
    ├── test_feature_gates.py
    ├── test_pipeline_e2e.py
    └── test_agents.py


careorbit-web/src/app/(dashboard)/
├── page.tsx                   # Dashboard home
├── medications/page.tsx
```

```
├── care-gaps/page.tsx
├── upload/page.tsx
├── chat/page.tsx
├── summary/page.tsx
├── family/page.tsx                  # NOW P0 — family dashboard
├── labs/page.tsx                    # NOW P0 — trend charts
├── adherence/page.tsx                   # NOW P0 — streak + bar charts
└── settings/subscription/page.tsx
```